Pusan National University

Computer Science and Engineering

Technical Report 2026-09

장기계획 문제를 위한 스킬 기반

Pokémon-Playing Agent

202155625 홍세민 semin73@pusan.ac.kr

202055602 조재우 nicewodn@pusan.ac.kr

지도교수 : 김원석

목 차

1. 서론 1

1.1. 연구 배경 1

1.2. 기존 문제점 2

1.3. 접근 방안 및 목표 4

1.3.1. 과제 구조화 및 연구 범위 4

1.3.2. 연구 목표 5

2. 관련 이론 및 기술 5

2.1. Pokémon Red 게임 환경 5

2.1.1. 게임 조작 및 복합 과제 5

2.1.2. 이벤트 기반 게임 진행 6

2.2. 강화학습 6

2.2.1. 강화학습 개요 6

2.2.2. PPO 7

2.2.3. 희소보상 7

2.2.4. Reward Shaping 8

2.3. 장기 계획(Long-Horizon Planning) 9

2.4. 대규모 언어 모델(LLM) 9

2.5. PyBoy 기반 게임 실행 환경 10

3. 연구 내용 11

3.1. Map 11
3.1.1. 지도 구조의 분리와 전체 구성 11
3.1.2. Global Map 12
3.1.3. Local Map 14
3.2. Skill 18
3.2.1. Navigation 19
3.2.2. Exploration 21
3.2.3. Battle 23
3.3. LLM Planner 26
4. 연구 결과 분석 및 평가 31
4.1. Exploration Skill 성능 분석 및 평가 31
4.2. Battle Skill 성능 분석 및 평가 33
4.3. 통합 실행 결과 35
5. 결론 및 향후 연구 방향 37
5.1. 결론 37
5.2. 향후 연구 방향 38
6. 구성원별 역할 및 개발 일정 39
7. 참고 문헌 39

1. 서론

1.1. 연구 배경

최근 인공지능은 단일 작업을 수행하는 모델을 넘어, 복잡한 환경을 인식하고 현재 상황을 판단하여 장기적인 목표를 수행하는 방향으로 발전하고 있다. 이러한 흐름은 인공지능이 가상 공간을 벗어나 실제 물리 세계에서 인지·판단·행동하는 피지컬 AI 로 이어지고 있다. 특히 NVIDIA 의 젠슨 황 CEO 가 CES 2025 기조연설에서 피지컬 AI 를 제시한 이후, 피지컬 AI 는 산업계와 학계의 핵심 키워드로 부상하였다.

시장 규모 역시 이러한 관심을 뒷받침한다. Statista 에 따르면 전 세계 AI 로봇 시장은 2020 년 약 320 억 달러에서 등락을 거친 뒤 2025 년 약 710 억 달러로 확대되었으며, 2030 년 약 1,290 억 달러, 2032 년 약 1,610 억 달러까지 성장할 것으로 전망된다 [1]. 범위를 피지컬 AI 로 좁혀도, Mordor Intelligence 는 해당 시장이 2025 년 약 51 억 달러에서 2031 년 약 349 억 달러로 연평균 37.5% 성장할 것으로 예측하였다 [2]. 이처럼 피지컬 AI 는 기술적 관심과 산업적 수요가 함께 확대되는 분야로 자리 잡고 있다.

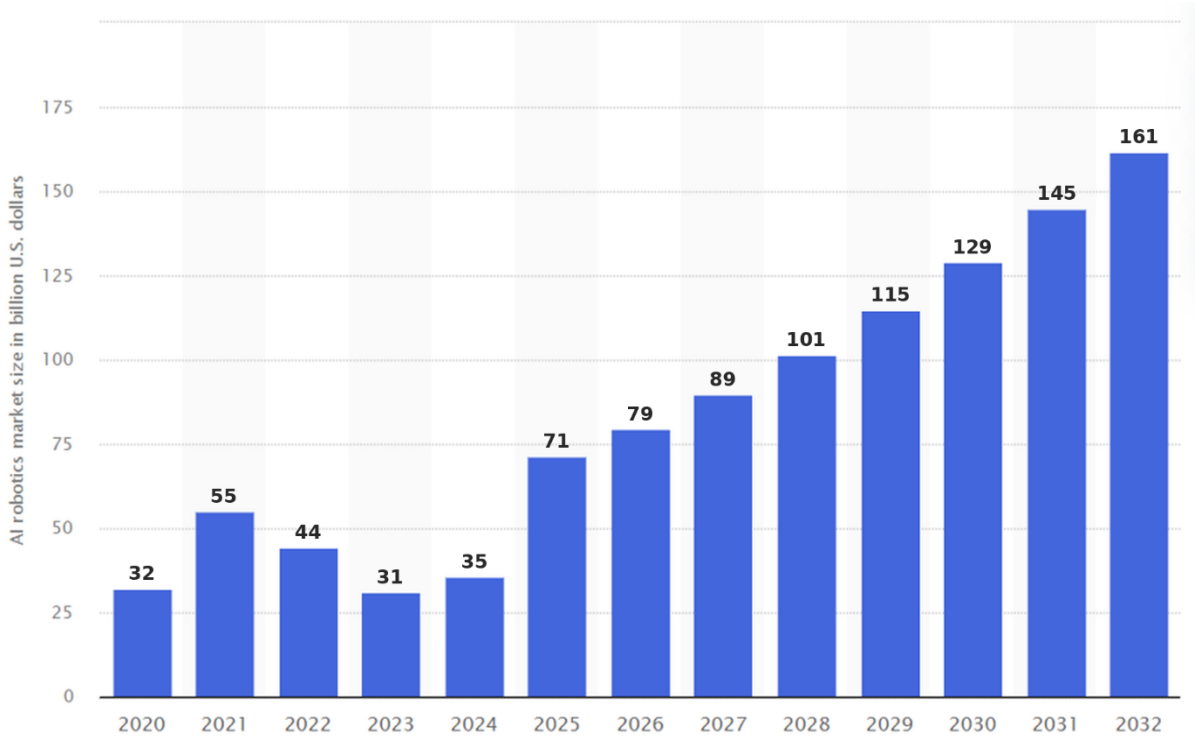


그림 1 Statista, "AI robotics market size worldwide 2020–2032"

그러나 피지컬 AI 를 실제 로봇에 적용하려면 먼저 학습 환경의 문제를 해결해야 한다. 강화학습 기반 로봇 정책은 수백만 회 이상의 시행착오를 필요로 한다. 이를 실제 환경

에서만 수행하면 막대한 시간과 비용이 들고, 장비 마모와 안전 위험, 매 시행마다 환경을 초기화해야 하는 부담이 따른다. 이에 따라 시뮬레이션에서 정책을 학습한 뒤 실제 로봇으로 전이하는 sim-to-real 방식이 대안으로 자리 잡고 있다. 또한 로봇이 실제로 수

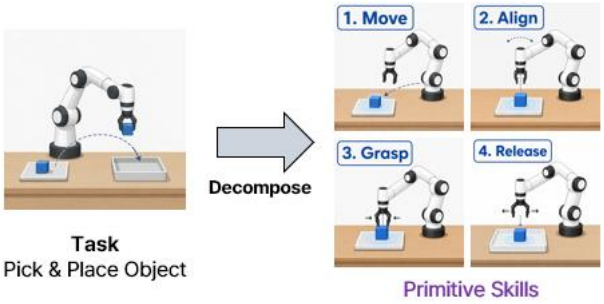


그림 2 로봇 Arm 의 Pick & Place 복합 과제

행해야 하는 작업 중에는 단일 동작이 아닌, 여러 하위 스킬을 순서대로 조합해야 하는 복합 과제도 존재한다. 예를 들어, "물건을 주워 옮기기"는 Move, Align, Grasp, Release 라는 하위 스킬들이 선행 조건에 따라 연결되어야 완수된다.

이러한 문제를 해결하는 구조를 검증하려면 실제 로봇보다 빠르고 안전하게 반복 실험할 수 있으면서, 장기 복합 과제의 특성을 그대로 갖는 환경이 필요하다. 이러한 장기 복합 과제의 구조가 "포켓몬스터" 게임에서도 동일하게 나타난다는 점을 확인하였다. 포켓몬스터를 진행하려면 맵 탐색, 배틀, NPC 상호작용 등 성격이 전혀 다른 기술을 번갈아 사용해야 한다. 또한 특정 배지나 기술을 획득해야만 새로운 지역으로 이동할 수 있어, 행동의 가능 여부가 선행 조건에 의존한다. 이는 로봇이 선행 조건을 만족시키며 여러 기술을 조합해 장기 목표를 달성하는 과정과 구조적으로 대응된다. 동시에 포켓몬스터는 에뮬레이터를 통해 빠르고 안전하게 반복 실험할 수 있으므로, 실제 로봇 실험에 앞서 계층형 의사결정 구조를 검증하는 시뮬레이션 실험환경으로 적합하다.

따라서 본 과제는 Pokémon Red 를 실험 환경으로 하여, LLM 기반 상위 플래너와 하위 스킬을 결합한 Pokémon-Playing Agent 를 개발하고, 이를 통해 로봇의 장기 복합 과제에도 적용 가능한 계층형 의사결정 구조를 제시하고자 한다.

1.2. 기존 문제점

'Pokémon Red'를 강화학습으로 해결하려는 대표적인 선행 연구로는 PufferLib 기반의 연구가 있으며, 하나의 정책이 게임 전체를 학습하는 단일 에이전트 방식을 사용하였다. Puffer 는 PPO 기반 에이전트로 첫 번째 체육관 관장을 꺾고 블루시티를 지나, 갈색시티까지 도달하였다 [3]. 또한 1,000 만 개 미만의 파라미터를 가진 정책으로 게임 클리어를

보고하였으나 [4], 이는 하나의 정책이 게임을 완주한 것이 아니라 일부 구간을 스크립트로 대체한 결과이다. 이러한 단일 에이전트 방식은 다음 두 가지 한계를 가진다.

첫째, 단일 정책으로는 여러 스킬이 연결된 **복합 과제를 스스로 수행하기 어렵다**. Puffer의 에이전트는 갈색시티까지 도달하였지만, 어떤 보상 설계나 실험 조건으로도 입구가 풀베기 나무로 막힌 갈색시티 체육관에는 들어가지 못하였다 [3]. 체육관에 들어가려면 풀베기를 배울 수 있는 포켓몬과 비전머신(HM01)을 얻고, 메뉴를 조작해 기술을 가르친 뒤, 나무 앞에서 올바르게 사용해야 한다. 그러나 이 행동들은 그 전까지 한 번도 필요하지 않았기 때문에, 에이전트는 보상을 통해 이를 학습할 기회를 얻지 못하였다. 이처럼 이동, 메뉴 조작, 필드 상호작용 등 성격이 다른 스킬이 순서대로 이어져야 하는 과제는 하나의 정책이 학습하기 어렵다. 실제로 Puffer 는 풀베기를 가르치고 필드에서 사용하는 과정을 스크립트로 처리하였다.

둘째, **희소 이벤트를 매번 학습하는 과정이 비효율적**이다. 'Pokémon Red'에는 길을 막고 잠든 잠만보를 포켓몬 피리로 깨우는 이벤트처럼, 진행에 필수적이지만 드물게 발생하는 이벤트가 존재한다. 이러한 이벤트는 발생 빈도가 낮아 일반적인 탐험 보상만으로는 학습 신호를 얻기 어렵다. 따라서 강화학습으로 학습시키려면 이벤트마다 별도의 보상을 설계해야 한다. 결국 단일 에이전트 방식에서는 새로운 희소 이벤트가 등장할 때마다 보상 설계와 재학습을 반복해야 하므로 비효율적이다.

이러한 한계로 인해 단일 에이전트 강화학습만으로는 'Pokémon Red'의 복합 과제와 희소 이벤트를 효율적으로 처리하기 어렵다.

1.3. 접근 방안 및 목표

1.3.1.과제 구조화 및 연구 범위

앞서 제시한 단일 에이전트 방식의 한계를 해결하기 위해, 본 연구는 'Pokémon Red'의 과제를 성격과 선후관계에 따라 구조화한다. 게임에는 이동, 전투, 퍼즐, 아이템 관리 등 다양한 과제가 존재하며, 이를 모두 한 번에 고려하여 에이전트를 설계하는 것은 지나치게 복잡하다. 따라서 과제들을 계층으로 정리하고, 가장 기본이 되는 과제부터 차례로 해결한다.

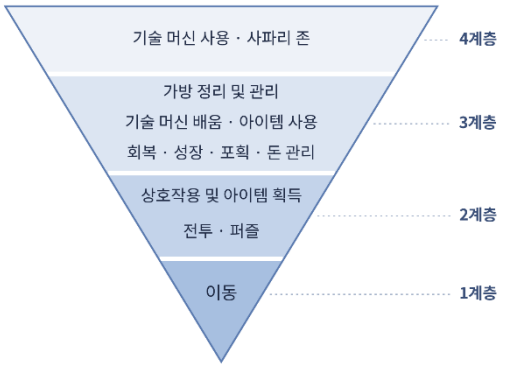


그림 3 'Pokémon Red' 과제의 계층 구조

그림 2 와 같이 게임 진행의 기반이 되는 이동을 1 계층에 두고, 이를 전제로 수행되는 과제를 상위 계층에 배치한다. 과제 간의 관계는 다음 두 가지로 구분한다.

1. **선행 관계**: 하위 계층의 과제가 해결되어야만 수행할 수 있는 관계이다. 예를 들어 기술머신을 획득해야 기술을 배울 수 있고, 기술을 배워야 필드에서 사용할 수 있다.
2. **동시 관계**: 하위 계층의 과제가 끝나지 않은 상태에서도 함께 발생할 수 있는 관계이다. 예를 들어 목적지로 이동하는 도중에도 야생 포켓몬이나 트레이너와의 전투가 발생할 수 있다.

상위 계층의 과제일수록 여러 하위 과제를 전제로 하고 고려해야 할 조건이 많아, 모든 계층을 한 번에 다루기에는 복잡도가 지나치게 높다. 따라서 본 연구는 게임 진행의 기반이 되는 1 계층의 이동과 2 계층의 전투를 우선 해결 대상으로 한다. 이동은 새로운 지역을 탐험하는 과제와 이미 알고 있는 맵에서 목표 지점까지 이동하는 과제로 성격이 나뉘므로, 이를 구분하여 다룬다. 이에 따라 본 연구는 탐험, 목표 지점 이동, 전투의 세 가지 과제를 해결 대상으로 한다. 그 외 상위 계층의 과제인 아이템 관리, 기술머신 사용 등은 본 연구의 범위에서 제외하며, 사전에 정의된 규칙 기반 스크립트로 처리한다.

1.3.2.연구 목표

본 연구의 최종 목표는 'Pokémon Red'와 같이 장기 과제가 중첩된 복합 게임 환경에서, 사용자의 개입 없이 게임을 진행하는 LLM 플래너와 스킬 기반의 **계층적 에이전트 구조**를 제시하고 그 가능성을 검증하는 것이다. 이를 위한 세부 목표는 다음과 같다.

- **[과제 구조화]** 단일 정책으로 다루기 어려운 게임 진행 과제를 **탐험 · 목표 지점 이동 · 전투**의 세 하위 과제로 분해하고, 다음 목표를 결정하는 상위 판단을 LLM 플래너가 담당하도록 분리한다.
- **[모듈별 최적 기법 적용]** 각 하위 문제의 특성(탐색 공간, 보상 밀도, 의사결정 주기)에 따라 서로 다른 해법을 선택적으로 적용한다.
- **[파이프라인 통합]** 분리된 모듈을 하나의 제어 흐름으로 결합하여, 외부 개입 없이 동작하는 자율 진행 구조를 구축한다.
- **[가능성 검증]** 통합된 에이전트가 실제 게임 환경에서 장기 과제를 수행할 수 있음을 실험적으로 확인한다.

2. 관련 이론 및 기술

2.1. Pokémon Red 게임 환경



그림 4 복합적인 과제로 이뤄진 Pokémon Red 환경

2.1.1. 게임 조작 및 복합 과제

'Pokémon Red'는 Game Boy 플랫폼에서 동작하는 RPG 게임으로, 상·하·좌·우 방향키와 A, B 버튼을 중심으로 게임을 조작한다. 기본적인 조작버튼을 조합하여 전투, 이동, 상호작용 및 메뉴 선택 등의 복잡한 조작을 수행해야 한다.

또한, 'Pokémon Red'의 게임 진행은 단일 행동 양식의 반복이 아닌, 성격이 다른 여러 과제가 연속적으로 발생하는 구조를 가진다. 그림 4 와 같이 플레이어가 새로운 지역을 탐험하고 목적지까지 이동하며, 그 과정에서 NPC 및 오브젝트와 상호작용하고 야생 포켓몬이나 트레이너와 전투를 수행한다. 즉, 에이전트가 주어진 **과제**에 따라 **적절한 행동 방식**을 선택하며 게임을 진행해야 한다.

2.1.2.이벤트 기반 게임 진행

'Pokémon Red'는 지역을 자유롭게 이동하며 전투하는 오픈월드의 방식이 아니라, 다수의 이벤트가 일정한 **선후관계**로 연결된 스토리 기반 게임이다. 목적지에 도착하더라도 선행 이벤트가 완료되지 않았다면 게임 진행이 막히며, 특정 이벤트를 완료해야 새로운 지역과 이벤트를 진행할 수 있게 된다.

다시 말해, 게임 내 같은 위치에 있더라도 특정 이벤트의 수행여부에 따라 다음에 수행해야 될 목표가 달라지므로 에이전트의 주기적인 메모리 갱신 및 장기적인 계획이 필요함을 의미한다.



그림 5 'Pokémon Red' 게임 초반 이벤트 진행 예시

2.2. 강화학습

2.2.1.강화학습 개요

강화학습(Reinforcement Learning, RL)은 에이전트가 환경과 상호작용하며 얻은 경험과 **보상**을 바탕으로 행동을 학습하는 방법이다. 그림 6 과 같이 에이전트는 현재 상태를 확인하고 행동을 선택하며, 환경은 그 결과로 다음 상태와 보상을 제공한다. 에이전트는 이 과정을 반복하면서, 각 상태에서 어떤 행동을 선택할지 결정하는 규칙인 정책(policy)을 학습한다.

이때 에이전트에게는 정답 행동이 주어지지 않으므로, 여러 행동을 직접 시도하고 그 결과로 얻은 보상을 비교하여 더 나은 행동을 찾아야 한다. 또한 당장의 보상뿐 아니라, 현재 행동이 이후에 가져올 보상까지 함께 고려하여 정책을 학습한다.

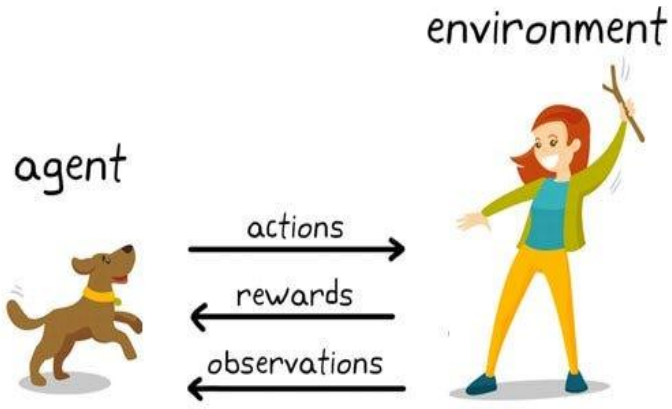


그림 6 강화학습에서 에이전트와 환경의 상호작용

2.2.2.PPO

PPO(Proximal Policy Optimization)는 에이전트의 정책을 학습하는 대표적인 강화학습 알고리즘이다. 에이전트는 환경과 상호작용하며 상태와 보상을 수집하고, 더 높은 보상으로 이어지는 행동을 더 자주 선택하도록 정책을 수정한다. 이때, 정책을 한 번 크게 변경하면 그동안 학습한 행동이 무너지거나 학습이 불안정해질 수 있다. 그래서 PPO 는 새로운 정책이 기존 정책에서 지나치게 멀어지지 않도록 한 번에 바뀌는 폭을 제한한다. 이를 통해 수집한 경험을 여러 번 재사용하면서도 정책을 안정적으로 갱신할 수 있다.

PPO 는 일반적으로 Actor 와 Critic 의 두 가지 구성 요소를 함께 사용한다. Actor 는 현재 상태에서 수행할 행동을 선택하고, Critic 은 해당 상태에서 앞으로 얻을 수 있는 보상을 예측한다. 실제로 얻은 결과를 Critic 의 예측과 비교하면 선택한 행동이 예상보다 좋은 결과를 만들었는지 판단할 수 있다. 에이전트는 이러한 평가를 바탕으로 Actor 와 Critic 을 반복해서 갱신하며 행동을 학습한다.

2.2.3.희소보상

희소보상(Sparse Reward)은 에이전트가 많은 행동을 수행하더라도 의미 있는 보상을 드물게 받는 문제를 말한다. 예를 들어 게임에서 최종 목표를 달성했을 때만 보상이 주어진다면, 목표에 도달하기 전까지 수행한 대부분의 행동에서는 학습에 필요한 정보를

얻기 어렵다. 이 경우 에이전트는 어떤 행동이 목표 달성에 도움이 되었는지 판단하기 힘들어진다.

환경이 넓고 목표 달성까지 필요한 행동이 많을수록, 에이전트가 우연히 보상을 발견할 가능성은 더욱 낮아진다. 무작위 행동만으로 목표를 찾으려면 많은 시행착오와 학습 시간이 필요하므로, 희소보상 환경에서는 새로운 상태를 효과적으로 탐색하고 학습에 필요한 경험을 확보하는 방법이 중요하다.

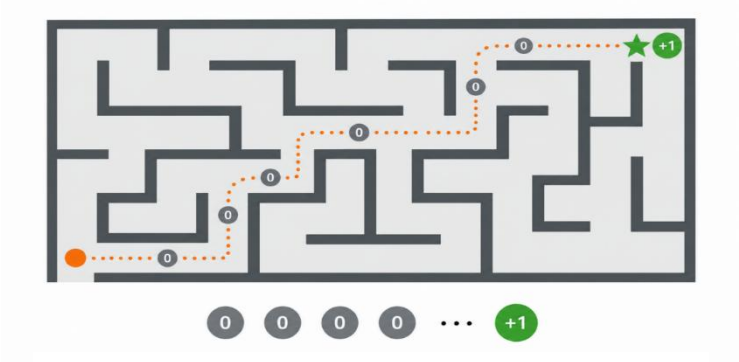


그림 7 희소보상 문제 예시

2.2.4.Reward Shaping

보상 형성(Reward Shaping)은 최종 목표에서 주어지는 보상 외에, 학습에 도움이 되는 중간 보상을 추가하는 방법이다. 목표에 가까워지는 과정에도 적절한 보상을 제공하면 에이전트가 행동의 결과를 더 쉽게 구분할 수 있으며, 의미 없는 탐색을 줄이는 데 도움이 된다.

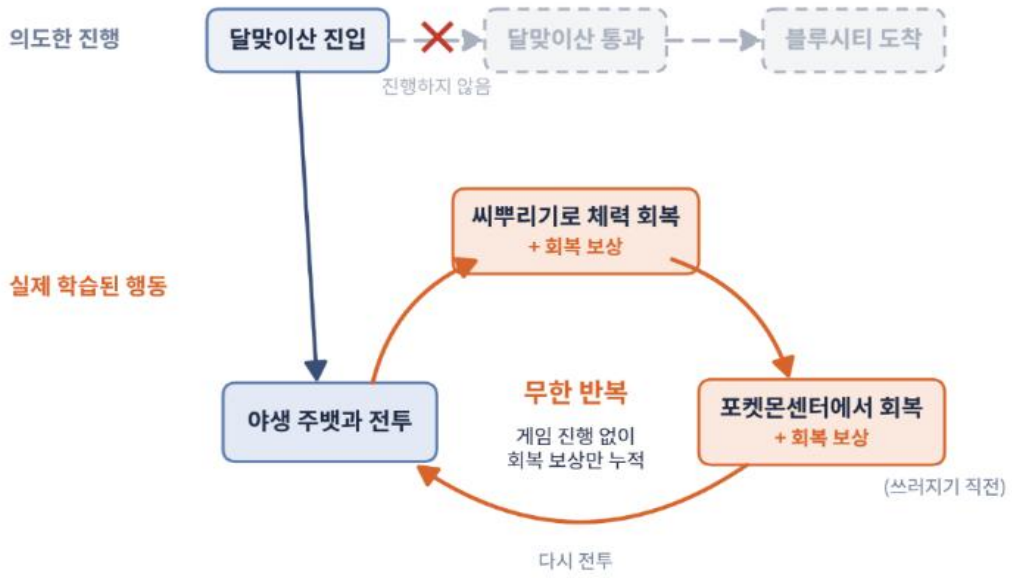


그림 8 Pokémon Red 의 보상 해킹 사례

그러나, 중간 보상을 잘못 설계하면 에이전트가 최종 목표 대신 중간 보상만 반복해서 얻는 행동을 학습할 수 있다. 이처럼 에이전트가 설계자가 의도한 목표가 아니라 보상 함수의 허점을 이용해 보상을 최대화하는 현상을 보상 해킹(Reward Hacking)이라 한다. 그림 8 은 'Pokémon Red'에서 실제로 관찰된 보상 해킹의 예시로, 체력 회복에 보상을 받은 에이전트가 달맞이산을 통과하지 않고 야생 포켓몬과의 전투와 회복만 반복하여 보상을 쌓는 전략으로 수렴하였다. 따라서 Reward Shaping 에서는 중간 보상이 최종 목표와 어긋나지 않도록 신중하게 설계해야 한다.

2.3. 장기 계획(Long-Horizon Planning)

장기 계획(Long-Horizon Planning)은 목표를 달성하기 위해 여러 단계의 행동과 의사결정을 장기간 연속적으로 수행해야 하는 문제를 의미한다. 단기적인 문제에서는 즉각적으로 좋은 행동을 선택하는 것으로 충분하지만, 장기 계획 문제에서는 현재 행동이 여러 단계 이후의 결과에 어떤 영향을 미치는 지까지 고려해야 한다.

강화학습에서는 목표 달성까지 필요한 행동의 수가 늘어날수록 어떤 행동이 장기적인 목표에 기여했는지 판단하기 어려워진다. 또한, 상태와 가능한 행동 순서의 조합이 증가하면서 탐색해야 할 경우의 수도 크게 늘어난다.

따라서 장기 계획 문제에서는 매 시점의 행동을 독립적으로 선택하기보다, 긴 과제를 여러 개의 중간 목표(Subgoal)로 나누거나 일정 기간 동안 수행되는 행동을 하나의 단위로 묶어 문제를 단순화한다.

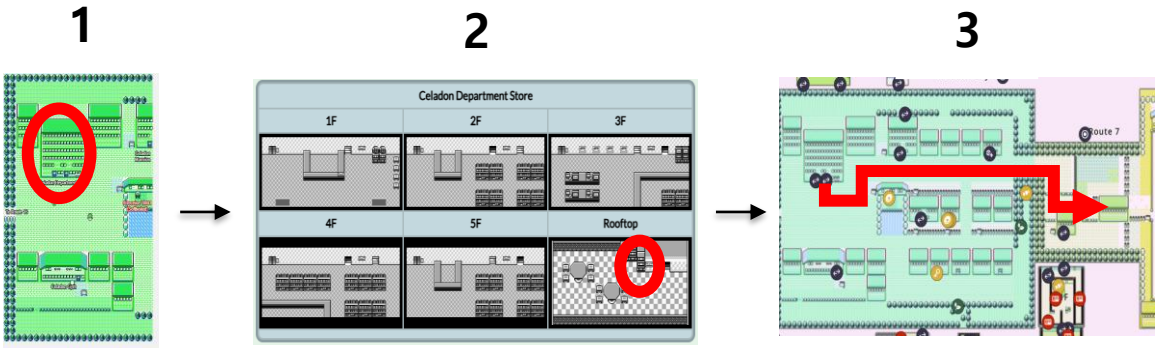


그림 9 장기계획 문제 예시

2.4. 대규모 언어 모델(LLM)

대규모 언어 모델(Large Language Model, LLM)은 많은 양의 텍스트를 학습하여 단어와 문장 사이의 관계를 파악하고, 주어진 입력에 이어질 내용을 생성하는 언어 모델이다. 사

용자가 지시문과 관련 정보를 제공하면, LLM 은 그 내용을 바탕으로 질문에 답하거나 여러 정보를 정리하고 다음에 수행할 작업을 제안할 수 있다.

LLM 은 정해진 규칙만 따르는 방식과 달리, 입력으로 주어진 상황을 종합하여 목표나 작업 순서를 제안할 수 있다. 이러한 특징은 여러 단계가 연결된 문제에서 현재 상황에 맞는 다음 목표를 결정하는 데 활용할 수 있다.

2.5. PyBoy 기반 게임 실행 환경

PyBoy 는 Game Boy 게임을 Python 에서 실행하고 제어할 수 있도록 만든 에뮬레이터이다. 일반적인 에뮬레이터처럼 게임 ROM 을 실행할 수 있으며, Python 코드를 통해 게임 화면과 메모리(RAM)에 저장된 내부 상태를 읽고 버튼 입력을 전달할 수 있다.

에이전트가 게임을 플레이하려면 현재 게임 상태를 확인하고, 선택한 행동을 다시 게임에 전달할 수 있어야 한다. PyBoy 는 이러한 게임과 에이전트 사이의 상호작용을 가능하게 한다. 따라서 에이전트가 실제 게임에서 제대로 행동하는지 확인할 수 있다.

3. 연구 내용

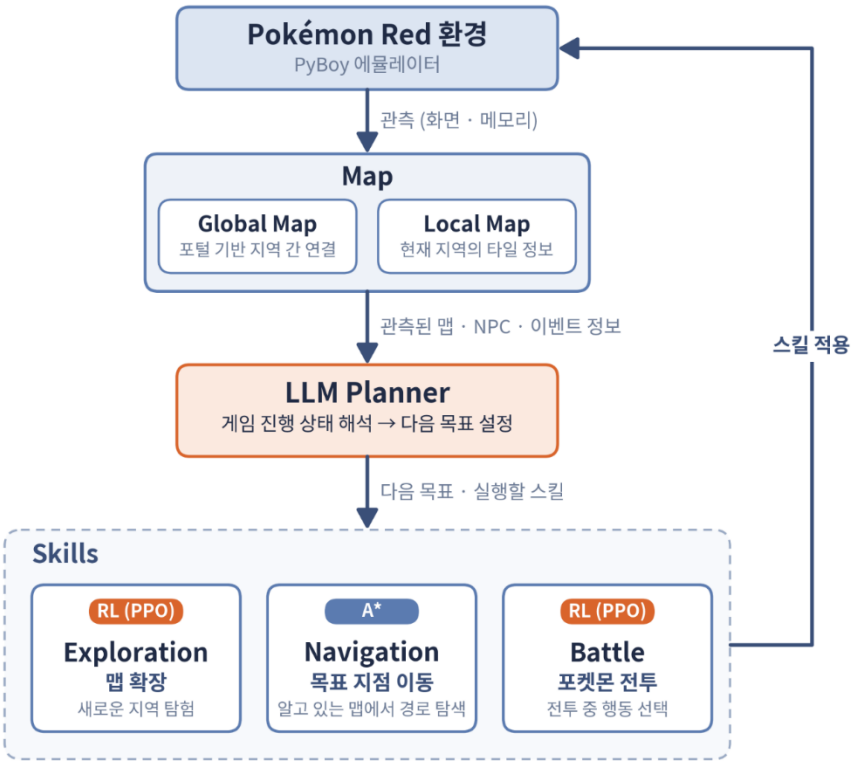


그림 10 전체 구조도

본 연구에서 제안하는 Pokémon-Playing Agent 는 그림 10 과 같이 Map, LLM Planner, Skills 의 세 부분으로 구성된다. Map 은 PyBoy 에서 읽은 화면과 메모리 정보를 누적하여 게임 공간을 지도로 관리하고, LLM Planner 는 이를 해석하여 다음에 도달할 목표를 결정하며, Skills 는 그 목표를 실제 행동으로 수행한다. 이는 서로 다른 성격의 과제를 하나의 정책으로 학습할 때 발생하는 간섭을 피하고, 강화학습으로 다루기 어려운 상위 판단을 별도 계층으로 분리하기 위한 구조이다.

3.1. Map

3.1.1.지도 구조의 분리와 전체 구성

본 연구에서는 게임 진행 과제를 탐험, 목표 지점 이동, 전투로 분리하였다. 각 모듈이 독립적으로 동작하더라도 게임 공간에 대한 정보는 공통으로 필요하다. 예를 들어 Navigation 은 목적지까지의 경로를 계산해야 하고, Exploration 은 아직 확인하지 않은 지역을 판단해야 하며, LLM Planner 는 계획을 세우기 위해 이동할 수 있는 맵과 NPC 목록 등 현재까지 관측된 정보를 알아야 한다. 따라서 본 연구는 에이전트가 관측한 공간 정보를 지도로 저장하여, 모든 모듈이 이를 함께 참조할 수 있도록 하였다.

지도는 Global Map 과 Local Map 으로 나누어 구성하였다. 'Pokémon Red'의 게임 공간은 하나의 연속된 지도가 아니라 마을, 도로, 건물, 동굴과 같은 여러 맵이 출입구를 통해 연결된 형태이다. 이러한 구조를 하나의 큰 격자로 표현하기는 어렵기 때문에, 맵 사이의 연결 관계는 그래프 형태의 Global Map 으로 표현하고, 각 맵 내부의 지형은 격자 형태의 Local Map 으로 표현하는 방식을 선택하였다.

다만 게임의 맵 번호는 경로 탐색의 단위로 사용하기에 적합하지 않은 경우가 있다. 'Pokémon Red'에는 실제로는 서로 오갈 수 없는 영역이 같은 맵 번호를 갖는 경우가 존재하여, 맵 번호만으로 경로를 계산하면 실행할 수 없는 경로가 만들어진다. 이를 해결하기 위해 맵이 아닌 출입구를 경로 탐색의 단위로 하는 Portal 을 추가하였다. 이에 대한 자세한 내용은 3.1.4 에서 설명한다.

관측의 정확도를 높이기 위해 게임의 정적 정보 일부를 참조 자료로 사용하였다. 맵 번호에 대응하는 맵 이름, 각 맵의 워프 좌표와 크기, 화면에 표시되지 않는 상호작용 오브젝트의 위치가 여기에 해당한다. 이러한 정보는 게임 진행에 따라 변하지 않으므로 미리 확보해 두어도 관측의 의미를 해치지 않는다. 다만 이 자료는 관측 결과를 보정하는 용도로만 사용하며, 지도에 미리 채워 넣지 않는다. 어떤 맵이 존재하고 어디로 연결되는지는 에이전트가 직접 관측한 것만으로 구성한다.

3.1.2.Global Map

Global Map 은 맵 사이의 연결 관계를 방향 그래프 $G = (V, E)$로 표현한다. 노드 $v \in V$는 게임이 맵 번호로 구분하는 개별 맵이고, 간선 $(u, v) \in E$는 에이전트가 실제로 걸어서 맵 u에서 맵 v로 이동한 것이 확인된 연결이다. 그래프 자료구조와 경로 탐색에는 NetworkX 라이브러리를 사용하였다.

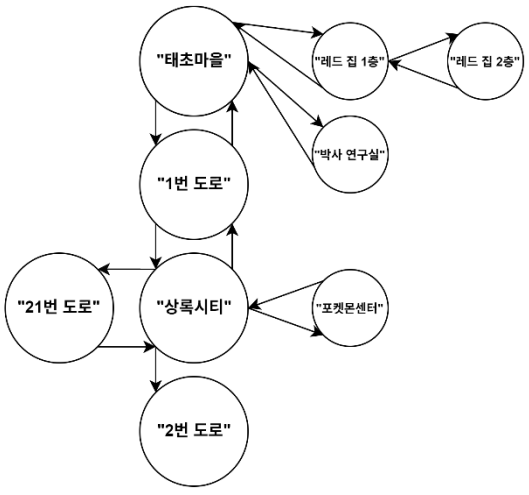


그림 11 GlobalMap 그래프 예시

노드는 맵 하나를 가리키는 동시에 해당 맵의 Local Map 을 담는 그릇 역할을 한다. 노드에는 게임에서 사용하는 맵 번호를 키로 하여 맵 이름과 Local Map 을 저장한다. 간선의 가중치는 모두 1 로 고정하였으므로, 두 맵 사이의 경로 비용은 맵 전환 횟수와 같다.

간선에는 출입구의 좌표를 저장하지 않는다. 어느 칸으로 나가서 어디에 도착하는지는 Local Map 에 기록되며, 이를 경로 계획에 사용할 수 있는 형태로 관리하는 것은 Portal 이 담당한다. Global Map 의 간선은 두 맵이 서로 인접해 있다는 사실만을 나타낸다.

구축

그래프는 미리 주어지지 않고 모두 에이전트가 직접 관측한 것만을 토대로 노드와 간선이 구성된다.

노드는 처음 보는 맵 번호가 확인되었을 때 빈 Local Map 과 함께 추가된다. 간선은 다음 근거를 토대로 추가한다. 첫째는 문이나 계단을 통한 전환으로 전환 직전의 위치와 바라보던 방향으로 추정한 출구 칸이 참조 자료에 기록된 워프 좌표 근처인 경우이다. 둘째는 도로 경계를 통한 전환으로, 해당 칸이 맵의 범위를 벗어나되 경계에 인접한 경우이다. 마을과 도로가 이어지는 지점이 여기에 해당한다.

두 조건 중 하나가 성립하면 양쪽 맵에 상대 맵의 이름을 기록하고 두 맵을 양방향 간선으로 연결한다. 한 번의 전환으로 역방향까지 함께 등록하는 이유는, 역방향을 따로 관측할 때까지 기다리면 왔던 길로 되돌아가는 경로를 계획할 수 없기 때문이다.

반대로 순간이동, 전투 패배 후 이동, 탈출 로프에 의한 전환은 간선을 만들지 않는다. 이러한 전환도 맵 번호를 바꾸지만 출구 칸이라는 근거가 없다. 간선의 의미를 걸어서

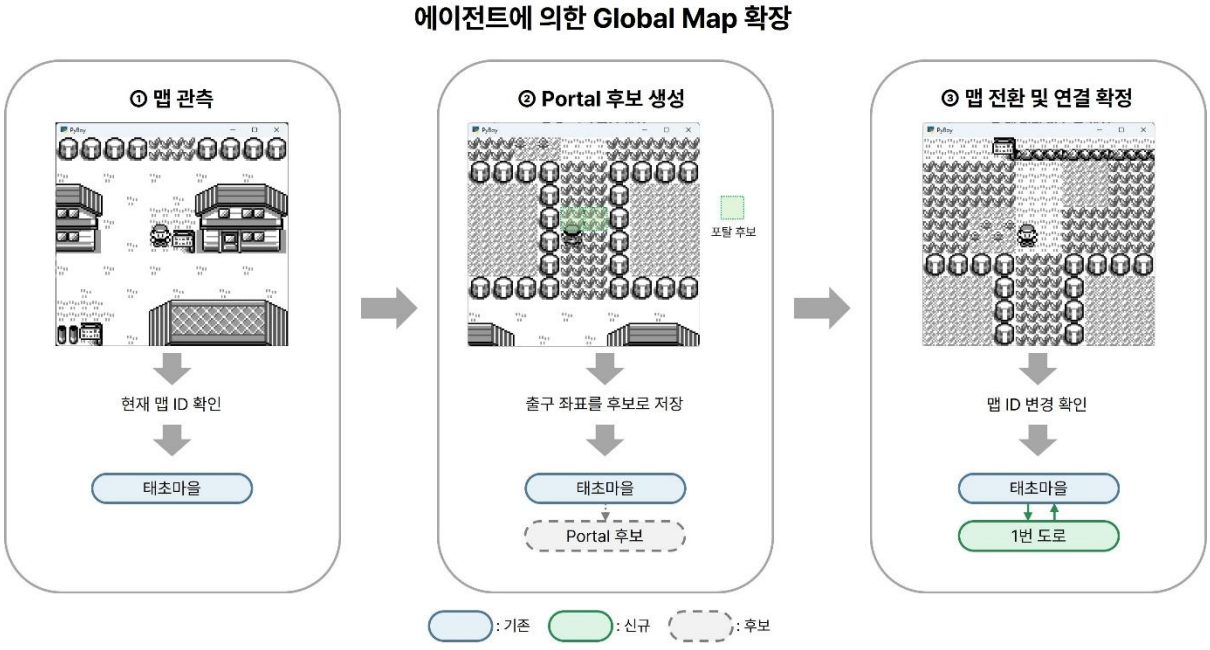


그림 12 관측에 의한 Global Map 예시

이동할 수 있다는 것으로 유지해야, 그래프에서 얻은 경로를 실제로 실행할 수 있기 때문이다.

사용

경로 탐색을 실제로 수행하는 것은 Global Map 이 아니라 3.1.4 의 Portal 그래프이다. 두 그래프는 노드의 단위가 다르고 답하는 질문도 다르다. Global Map 은 노드가 맵이며 어떤 맵들이 서로 인접해 있는지에 답하고, Portal 그래프는 노드가 개별 출입구이며 목표 맵까지 어느 출입구를 거쳐 가야 하는지에 답한다. 이러한 전제 위에서 Global Map 은 다음 세 가지 용도로 사용한다.

첫째, Local Map 의 저장소이다. 매 스텝의 화면 관측은 Global Map 을 거쳐 해당 노드의 Local Map 에 반영된다. 맵별 격자가 흩어지지 않고 하나의 그래프 안에 모이도록 하는 것이 이 구조의 기본 역할이다.

둘째, 맵 단위의 거리 추정에 사용한다. 목적지까지 경유할 맵의 목록을 제공하며, 간선 비용이 모두 1 이므로 그 길이는 맵 전환 횟수가 된다. 이는 전체 경로를 실행하기 위해서가 아니라 후보를 선택하기 위해 사용한다. 관측한 여러 포켓몬센터 중 어디가 가장 가까운지 판단할 때가 여기에 해당하며, 목적지가 정해진 뒤의 실제 이동 경로는 Portal 그래프가 계산한다.

셋째, LLM Planner 의 입력이 된다. 그래프에 등록된 맵의 목록과 인접 관계가 프롬프트에 그대로 포함되며, Planner 는 이 목록 안에 있는 맵만 목표로 지정할 수 있다. 즉 Global Map 은 에이전트가 현재까지 알고 있는 범위를 그대로 나타내며, 이 범위는 Exploration 이 새로운 맵을 발견할 때마다 넓어진다.

3.1.3.Local Map

Local Map 은 맵 하나의 지형을 담는 격자로, 좌표를 키로 하고 Cell 을 값으로 하는 딕셔너리를 사용하여 구현하였다. 키는 게임 메모리에서 사용하는 맵 내부 좌표를 그대로 사용한다. 관측은 사전에 누적하되, 경로 탐색이나 정책 입력과 같이 격자 형태가 필요한 경우에는 그 시점에 행렬로 변환하여 사용한다.

격자 전체를 고정 크기 배열로 잡지 않고 관측한 칸만 저장하였다. 에이전트는 맵의 크기와 형태를 미리 알지 못한 상태에서 관측을 시작하므로, 격자의 범위를 사전에 정할 수 없기 때문이다. 관측한 칸만 저장하면 에이전트가 이동하는 대로 격자가 넓어진다.

화면에서 격자로의 변환

게임 화면은 20×18 타일로 구성된다. 그러나 'Pokémon Red'에서 플레이어는 한 번에 두 타일씩 이동하며, 메모리에 기록되는 좌표도 이 단위를 따른다. 따라서 격자의 한 칸은 타일 하나가 아니라 2×2 타일 묶음으로 구성하였다. 타일 단위로 격자를 만들면 좌표의 절반이 실제로는 도달할 수 없는 칸이 되어, 경로 탐색 결과가 실제 조작과 어긋나기 때문이다. 이 2×2 묶음을 이동칸이라 하며, 화면 한 장은 10×9 이동칸에 해당한다.

플레이어는 화면에서 항상 (4, 4) 이동칸에 표시된다. 따라서 플레이어의 맵 좌표가 (px, py)일 때, 화면의 (cx, cy) 칸은 맵 좌표 (px + cx − 4, py + cy − 4)에 대응한다. 관측이 일어날 때마다 이 90 개의 칸을 확인하여 새로 저장하거나 기존 값을 갱신하며, 화면 밖으로 벗어난 칸은 삭제하지 않으므로 격자는 에이전트가 지나온 경로를 따라 누적된다.

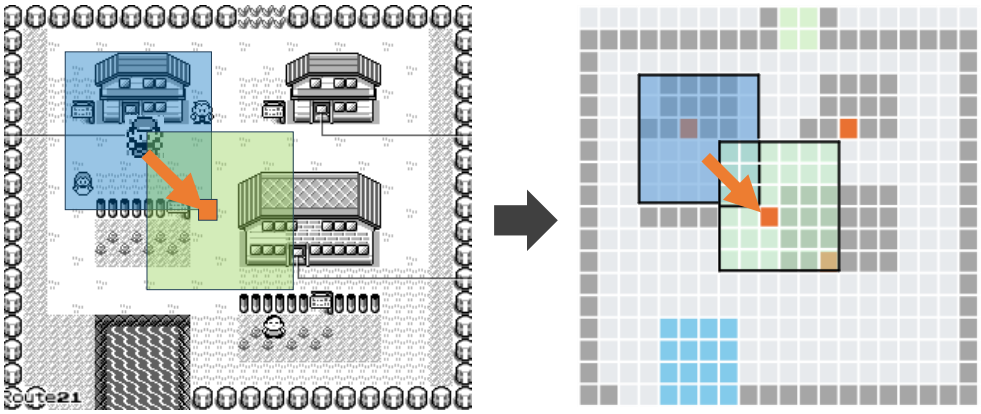


그림 13 태초마을 LocalMap 예시

Cell 의 구성

항목	내용
지형 종류	일반 통행, 벽, 물, 풀숲, 네 방향의 턱, 워프의 여덟 가지
출입구 이름	맵 전환이 확인되었을 때 기록하는 연결된 맵의 이름
워프 후보	화면 타일만으로 문이나 계단으로 판단되었는지 여부
상호작용 대상	NPC, 오브젝트, 풀베기 나무의 구분
NPC 이름	관측된 NPC 의 이름
방문 여부·횟수	에이전트가 해당 칸을 밟았는지와 그 횟수
경계 표시	아직 관측하지 않은 칸과 인접해 있는지 여부

표 1 Cell 의 구성

각 칸에 저장하는 항목은 표 1 과 같다. 여기에는 두 가지 설계 판단이 포함된다.

첫째, 통행 가능 여부를 참·거짓으로 저장하지 않고 지형의 종류를 저장하였다. 같은 칸이라도 에이전트가 보유한 능력에 따라 통행 여부가 달라지기 때문이다. 물은 파도타기를 사용할 수 있을 때만, 풀베기 나무는 풀베기를 사용할 수 있을 때만 지날 수 있으며, 턱은 방향이 맞을 때만 넘을 수 있다. 통행 여부를 미리 계산하여 저장하면 새로운 비전 머신을 얻을 때마다 이미 관측한 지도를 다시 확인해야 한다. 따라서 격자에는 지형만 저장하고, 통행 판정은 경로 탐색 시점에 당시의 능력과 함께 수행한다.

둘째, 워프 후보와 확정된 워프를 구분하여 저장하였다. 'Pokémon Red'의 맵 전환 지점은 두 종류로 나뉜다. 하나는 문이나 계단처럼 특정한 타일로 표시되는 경우이고, 다른 하나는 마을과 도로가 이어지는 지점처럼 별도의 표시 없이 맵 경계를 넘는 것만으로 전환이 일어나는 경우이다. 후자는 화면 타일만으로는 알아볼 수 없으며, 전자 역시 타일만으로는 어디로 연결되는지 알 수 없고 실제로 밟아야 확정된다.

따라서 관측만으로 표시하는 후보와, 통과하여 맵 전환이 확인된 워프를 따로 관리한다. 후보는 아직 들어가 보지 않은 출입구의 위치를 파악하는 데 사용하고, 확정된 워프는 Portal 을 구성하는 근거로 사용한다. 다만 타일 모양만으로 판정하면 포켓몬센터의 장식용 문처럼 실제로는 이동할 수 없는 칸까지 포함되므로, 워프 후보는 참조 자료에 출입구가 기록된 좌표 근처에서만 표시한다.

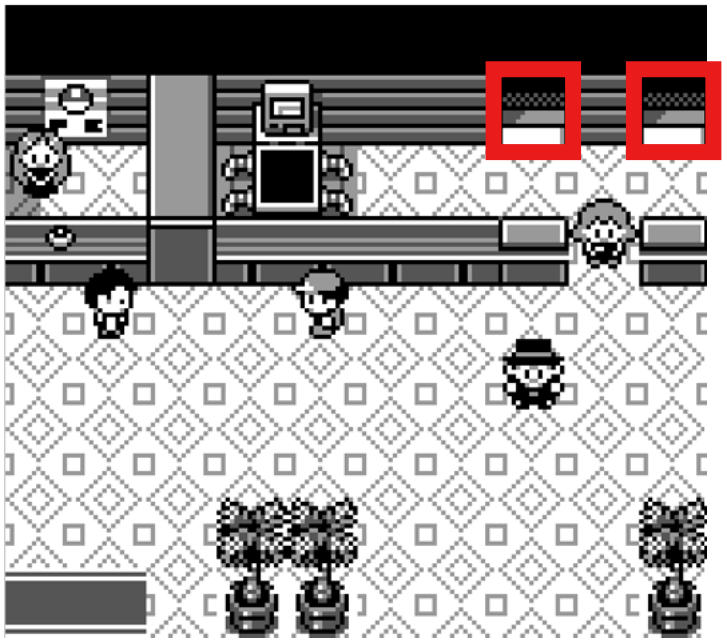


그림 14 포켓몬스터 장식용 문

NPC 이름을 두 가지 형태로 기록하는 것도 같은 이유에서이다. 하나는 좌표와 함께 저장하는 목록으로, 화면에서 사라지면 함께 삭제되며 대화를 위한 경로 탐색의 목표 좌표로 사용한다. 다른 하나는 해당 맵에서 한 번이라도 관측한 이름의 집합으로, 좌표를 갖지 않으며 삭제하지 않는다. NPC 는 이동하므로 과거의 좌표를 그대로 신뢰할 수 없지만, 그 맵에 해당 NPC가 존재한다는 사실은 계속 유효하기 때문이다. 후자는 LLM Planner 에게 어느 맵에 어떤 NPC 가 있는지를 전달하는 데 사용한다.

구축

Local Map 은 매 스텝의 화면 관측을 누적하여 구성된다. 화면에서 읽은 타일의 종류로 각 이동칸의 지형을 판정하고, 스프라이트 정보로 해당 칸에 NPC 나 오브젝트가 있는지를 함께 기록한다. 지형은 관측할 때마다 새로 갱신하므로, NPC 가 이동하면 다음 관측에서 해당 표시가 자동으로 사라져 일시적인 방해물이 지도에 남지 않는다. 다만 확정된 워프 표시는 화면 재판정으로 덮어쓰지 않으며, 워프 후보도 한 번 표시되면 유지한다. 화면 경계에서는 2×2 묶음의 판정이 관측마다 달라질 수 있기 때문이다.

중요한 것은 언제 관측하지 않는가이다. 격자는 누적되므로 한 번 잘못 기록된 칸은 계속 남는다. 따라서 화면이 해당 맵의 지형을 정확히 보여준다고 확신할 수 없는 구간에서는 관측을 수행하지 않는다. 전투 중에는 화면이 전투 장면으로 바뀌어 있고, 대화나 스크립트가 진행 중일 때는 텍스트 상자가 배경 타일을 가린다. 이동 중 화면 스크롤이 타일 경계에 정렬되지 않은 순간에는 읽어 낸 타일이 실제 위치와 어긋난다. 맵 전환 직후에는 맵 번호가 좌표와 화면보다 먼저 갱신되어 이전 맵의 정보가 새 맵의 격자에 기

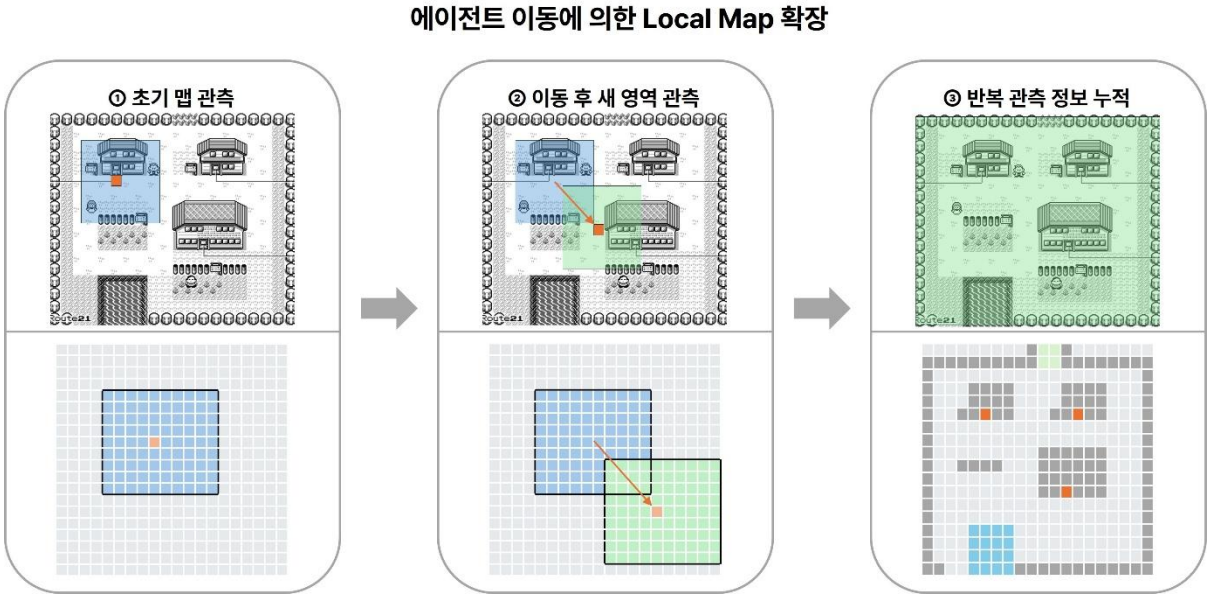


그림 15 관측에 의한 Local Map 예시

록될 수 있다. 이러한 경우에는 모두 관측을 건너뛰며, 같은 자리에 머무는 동안에는 화면이 정지한 프레임에서만 다시 읽는다.

사용 및 용도

첫째, 경로 탐색이다. 사전 형태의 격자를 숫자 행렬과 원점 좌표의 쌍으로 변환하여 A*에 전달한다. 이때 지형 종류와 현재 사용할 수 있는 비전머신을 함께 고려하여 각 칸의 통행 여부와 비용을 결정하며, NPC 나 오브젝트가 있는 칸은 벽으로 취급한다. 관측하지 않은 칸은 별도로 표시하여 경로가 그곳을 지나지 않도록 한다.

둘째, 탐험 범위의 판단이다. 각 칸의 방문 여부와 경계 표시가 아직 확인하지 않은 지역을 판별하는 근거가 된다. 탐험의 종료 시점을 판정하는 방식은 3.4 에서 다룬다.

셋째, Exploration 정책의 관측 입력이다. 플레이어를 중심으로 고정 크기의 영역을 잘라 지형 종류별 이진 평면으로 변환한 것이 정책의 격자 입력이 된다. 미관측, 통행, 벽, 물, 나무, 워프, NPC, 방문의 여덟 가지를 각각 한 장의 평면으로 구성한다.

넷째, 상위 계층에 대한 정보 제공이다. 워프 후보와 확정된 워프는 Portal 을 구성하는 근거가 되며, 맵별로 관측한 NPC 이름 목록은 LLM Planner 의 프롬프트로 전달되어 목표를 선택하는 근거가 된다.

3.2. Skill

스킬	해결 과제	적용 방안
Navigation	관측한 지도 위에서 목표 지점까지의 이동	A* 탐색
Exploration	미관측 지역으로의 진출과 지도 확장	PPO
Battle	전투 중 매 턴의 기술 사용과 교체 선택	PPO

표 2 세 Skill 의 비교

본 연구의 에이전트는 게임 진행을 세 가지 하위 과제로 나누고 각각을 하나의 Skill 로 구현하였다. Navigation 은 이미 관측한 지도 위에서 목표 지점까지 이동하고, Exploration 은 아직 관측하지 않은 지역으로 진출하여 지도를 넓히며, Battle 은 전투가 시작되었을 때 매 턴의 행동을 선택한다.

표 2 와 같이 세 과제에 서로 다른 방법을 적용한 것은 과제의 성격이 다르기 때문이다. Navigation 은 판단에 필요한 정보가 모두 갖추어져 있어 목표까지의 최단 경로를 계산으로 구할 수 있으므로, 학습을 적용하더라도 나아질 여지가 없다. 반면 Exploration 은 아직 보지 못한 지역의 가치를 미리 평가할 수 없고, Battle 은 상대의 선택과 확률적인 결과 때문에 최선의 수를 확정할 수 없다. 두 과제는 최선의 행동을 규칙으로 기술하기 어려워 시행착오를 통해 학습해야 하므로 강화학습을 적용하였다.

세 스킬은 서로의 결과 위에서 동작한다. Exploration 이 넓힌 지도가 있어야 Navigation 이 경로를 계획할 수 있고, Navigation 이 이동한 위치에서 Exploration 이 다시 새로운 지역을 관측한다. Battle 은 두 스킬이 실행되는 도중 언제든 시작되며, 전투가 끝나면 중단되었던 스킬이 이어서 실행된다.

3.2.1.Navigation

접근 방법

Navigation 은 LLM Planner 가 지정한 목표를 실제 이동으로 옮기는 역할을 담당한다.

이 과정은 전역과 지역의 두 층으로 나누어 수행한다. 맵 내부의 이동은 격자 위에서 어느 칸을 밟을 것인지의 문제이지만, 맵과 맵은 좌표로 이어져 있지 않고 출입구를 통해서만 연결되므로 같은 좌표 평면에서 다룰 수 없기 때문이다. 전역 층에서는 목표 맵까지 거쳐야 할 출입구의 순서를 정하고, 지역 층에서는 현재 위치에서 다음 출입구까지의 실제 경로를 구한다. 두 층 모두 A* 탐색을 사용하되, 휴리스틱은 전역 층에서 남은 최소 맵 전환 횟수를, 지역 층에서 목표 칸까지의 맨해튼 거리를 기준으로 정의하였다.

경로 탐색 과정

전역 탐색은 Portal 그래프 위에서 수행한다. 현재 위치와 목표 맵을 가상의 시작 노드와 끝 노드로 연결하되, 현재 위치에서 실제로 걸어갈 수 있는 출입구만 시작 노드와 연결한다. 목적지가 현재 맵 안에 있는 경우에는 전역 탐색을 건너뛰고 지역 탐색만 수행한다.

지역 탐색은 딕셔너리를 행렬로 변환한 뒤 다음 출입구까지의 경로를 구한다. 각 칸의 통행 여부는 저장된 지형 종류와 그 시점에 사용할 수 있는 비전머신을 함께 고려하여 판정한다. 물은 파도타기를, 풀베기 나무는 풀베기를 사용할 수 있을 때만 통과 가능한 칸이 되며, NPC 나 오브젝트가 있는 칸은 벽으로 취급한다. 턱은 방향이 맞을 때만 통과

할 수 있고, 이 경우 그 너머의 착지 칸으로 두 칸을 한 번에 이동하는 것으로 처리한다. 한편 아직 관측하지 않은 칸은 지형을 알 수 없으므로 제외하며, 목적지가 아닌 워프 칸도 밟는 순간 맵이 전환되어 계획한 경로가 무효가 되므로 제외한다.

이렇게 얻은 좌표의 나열은 이웃한 두 좌표의 차이를 이용해 방향키 입력으로 변환한다. 실제 이동은 이 과정의 반복으로 이루어지며, 출입구를 통과하여 맵이 전환되면 새로운 맵에서 다시 지역 탐색을 수행한다. 전역 탐색이 제시한 출입구 순서를 모두 소진하면 목표 맵에 도착한 것이다. 목표가 특정 NPC 인 경우에는 해당 NPC 앞까지 이동한 뒤 그 방향을 바라보는 단계가 추가된다.

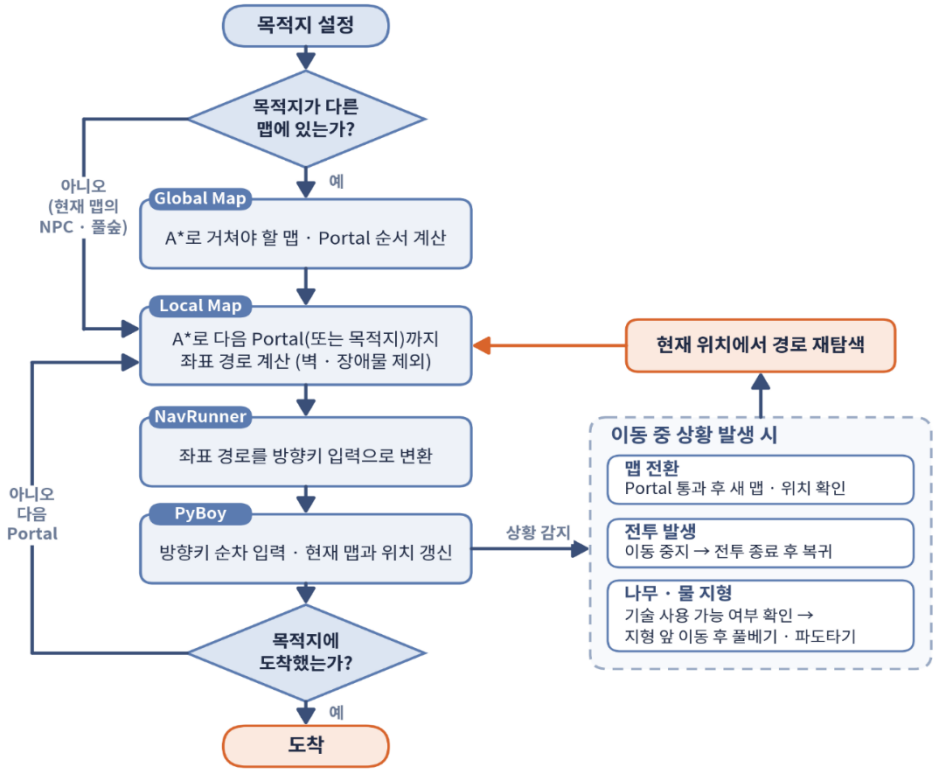


그림 16 Navigation Workflow Diagram

실행 및 예외 처리

계획한 경로를 끝까지 실행할 수 있는 경우는 많지 않으므로, 이동 중 발생하는 사건에 따라 다음과 같이 처리하였다.

전투가 시작되면 이동을 즉시 중단하고 Battle 에 처리를 맡기며, 전투가 끝난 뒤 그 시점의 위치에서 경로를 다시 계산한다. 대화가 발생한 경우에도 남은 경로를 버리고 다시 계산한다. 게임 스크립트가 대화 도중 플레이어를 다른 위치로 이동시키는 경우가 있기

때문이다. 다만 대화 후에도 위치가 변하지 않는다면 해당 NPC 가 길을 막고 있다는 의미이므로, 일정 횟수를 넘으면 이동을 중단한다.

앞을 막고 있는 것이 풀베기 나무나 물인 경우에는 기술을 사용하여 지형을 변경한다. 기술 사용은 메뉴를 여러 단계 조작해야 하는 정해진 절차이므로 별도의 규칙 기반 처리에 맡겼으며, Navigation 은 처리가 끝난 뒤 변경된 지형 위에서 경로를 다시 계산한다. 파도타기를 사용한 경우에는 물 건너편이 새로운 미관측 지역이 되므로, 해당 맵의 탐험 완료 표시도 함께 해제한다.

경로 탐색은 두 층 모두에서 실패할 수 있다. 관측한 출입구만으로 목표 맵까지 연결이 없거나, 다음 출입구까지의 격자 경로가 없는 경우이다. 두 경우 모두 이동을 중단하고 실패 사유를 기록하며, 이 기록은 다음 계획을 요청할 때 LLM Planner 의 입력으로 전달된다.

이처럼 재계산이 자주 발생하는 것은 지도가 미리 주어진 것이 아니라 관측으로 누적되기 때문이다. 계획 시점의 지도와 실행 중의 지도는 같지 않으므로, Navigation 은 한 번 세운 계획을 유지하는 대신 상황이 변할 때마다 현재의 지도와 위치를 기준으로 다시 계산하는 방식을 사용하였다.

3.2.2.Exploration

접근 방법

Exploration 은 아직 확인하지 않은 지역의 구조를 파악하고 새로운 맵을 발견하는 역할을 담당한다. Navigation 이 이미 관측한 지도 위에서 동작하는 것과 달리, Exploration 은 지도 자체를 넓히는 것이 목적이므로 매 순간 화면에 보이는 범위만을 근거로 다음 이동 방향을 결정해야 한다.

이때 맵의 역할에 따라 바람직한 행동이 다르므로, 그림 13 과 같이 맵을 세 가지로 구분하였다. 에피소드가 시작된 맵을 기준 맵인 Target 으로 두고, Target 에 직접 연결된 건물은 Internal, 그 밖의 도로나 마을, 동굴 등은 External 로 분류한다. Target 과 Internal 에서는 내부를 빠짐없이 확인하는 것이 목표이지만, External 은 새로운 지역이 있다는 사실만 확인한 뒤 되돌아오는 것이 목표이다.

관측과 행동

관측 상태에는 게임 화면과 플레이어 주변의 지도 정보인 Tracking Mask 를 사용한다. 게임 화면은 Game Boy 의 명암 정보를 한 채널로 축소하여 입력한다. Tracking Mask 는 플레이어를 중심으로 한 10×9 칸 범위에서 관측한 좌표, 직접 방문한 좌표, 충돌로 확인된 장애물, 실제로 통과한 워프 타일, NPC 와 오브젝트를 각각 채널로 표현한 것이다. 맵의 전체 구조나 아직 관측하지 않은 지형은 제공하지 않는다. 정책이 이미 알고 있는 지도를 보고 판단하는 것이 아니라, 관측 범위 안에서 다음에 확인할 방향을 스스로 선택하도록 하기 위함이다.

이 외에 현재 맵의 역할을 나타내는 Map Scope 와 플레이어가 바라보는 방향, 직전 행동과 그 이동 결과를 함께 입력한다. 행동은 상·하·좌·우 네 방향의 이동으로 구성하며, 한 번 선택한 방향키 입력은 캐릭터가 한 칸을 이동할 수 있도록 일정한 시간 간격으로 실행한다.

화면과 Tracking Mask 는 각각 CNN 으로 처 리하고, Map Scope 에 따라 지도 특징을 조절하는 FiLM 을 적용한다. 이후 모든 특징을 결합하여 LSTM 에 전달하며, Actor 는 네 방향 행동의 확률을 출력하고 Critic 은 현재 상태의 기대 보상을 추정한다. LSTM 은 최근 이동 방향과 충돌, 맵 전환의 흐름을 기억하며 에피소드가 끝나면 초기화된다.

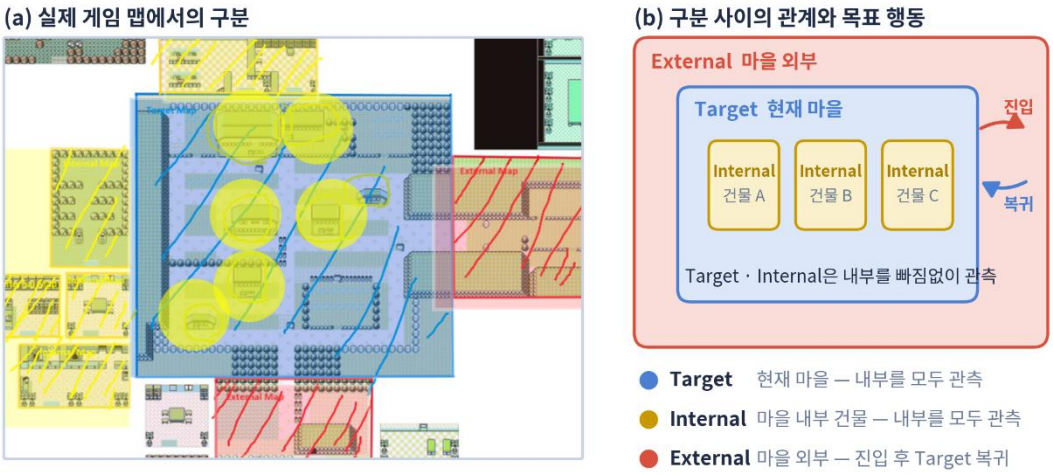


그림 17 Map Scope 구분과 각 구분에서의 목표 행동

보상 설계

탐험 보상은 Map Scope 에 따라 서로 다른 행동을 학습하도록 설계하였다. Target 과 Internal 에서는 처음 방문한 좌표와 새로 관측한 영역에 보상을 지급하고, 새로운 Internal 및 External 맵에 진입했을 때 추가 보상을 지급한다. External 에 진입한 뒤

Target 으로 복귀한 경우에도 보상을 지급하여, 외부 지역을 확인하고 돌아오는 행동을 유도하였다.

학습 환경

학습 환경은 PyBoy와 PufferLib를 이용하여 구성하였다. 학습은 병렬로 실행되며, 에피소드가 시작될 때 사전에 설정한 여러 저장 상태 중 하나를 무작위로 선택한다. 특정 지역의 구조에 학습이 치우치는 것을 방지하기 위함이다.

탐험 도중 전투가 발생하면 이동 경험의 수집이 중단되고 보상 계산이 불안정해질 수 있다. 이를 방지하기 위해 학습 환경에서는 야생 포켓몬과의 조우를 비활성화하고, 트레이너 전투가 발생하면 자동으로 종료하도록 하였다. 이에 따라 Exploration은 맵 관측과 이동 경로 학습에 집중한다.

실행

실제 게임에서는 학습과 동일한 방식으로 PyBoy 의 화면과 메모리 값을 관측 상태로 변환한다. 정책이 선택한 방향은 학습 환경과 같은 입력 시간으로 실행되며, 이동 결과는 다음 관측에 반영된다.

3.2.3.Battle

접근 방법

Battle 은 전투가 시작되었을 때 매 턴의 행동을 선택하는 역할을 담당한다. Navigation 과 Exploration 이 공간을 다루는 것과 달리, Battle 은 포켓몬의 체력과 타입, 상태 이상 등 게임 내부 상태를 근거로 판단한다.

전투 행동을 학습하기 위해 Pokémon Showdown 과 poke-env 를 이용하여 1 세대 전투 규칙을 기반으로 하는 별도의 학습 환경을 구성하였다. 실제 게임에서 전투를 반복하려면 이동과 대화 등 전투 외의 과정을 함께 거쳐야 하므로 학습에 필요한 경험을 충분히 수집하기 어렵기 때문이다.

관측과 행동

관측 상태에는 현재 전투 중인 아군과 상대 포켓몬의 체력, 타입, 상태 이상, 능력치 변화와 기술 정보를 포함하였다. 여기에 교체 가능한 아군 포켓몬과 전투 중 확인된 상

대 포켓몬의 정보도 함께 사용한다. 상대의 정보는 전투 중 드러난 부분만 알 수 있으므로, 관측된 범위 내에서 판단하도록 하였다.

행동은 파티 슬롯에 해당하는 6 개의 교체 행동과 현재 포켓몬이 보유한 4 개의 기술 행동으로 구성하여 총 10 개이다. 이 중 현재 전투에서 사용할 수 있는 행동만 선택 가능하며, 기절한 포켓몬이나 사용할 수 없는 기술은 선택 대상에서 제외한다.

10 개의 행동은 정책 내부에서 두 단계로 나누어 선택한다. 첫 번째 단계에서는 교체, 공격 기술, 상태 기술 중 수행할 행동의 종류를 정한다. 두 번째 단계에서는 선택한 종류에 해당하는 후보를 비교하여 실제 행동을 결정한다. 공격 기술을 선택한 경우에는 사용 가능한 공격 기술 중 하나를, 교체를 선택한 경우에는 교체 가능한 파티 포켓몬 중 하나를 고른다.

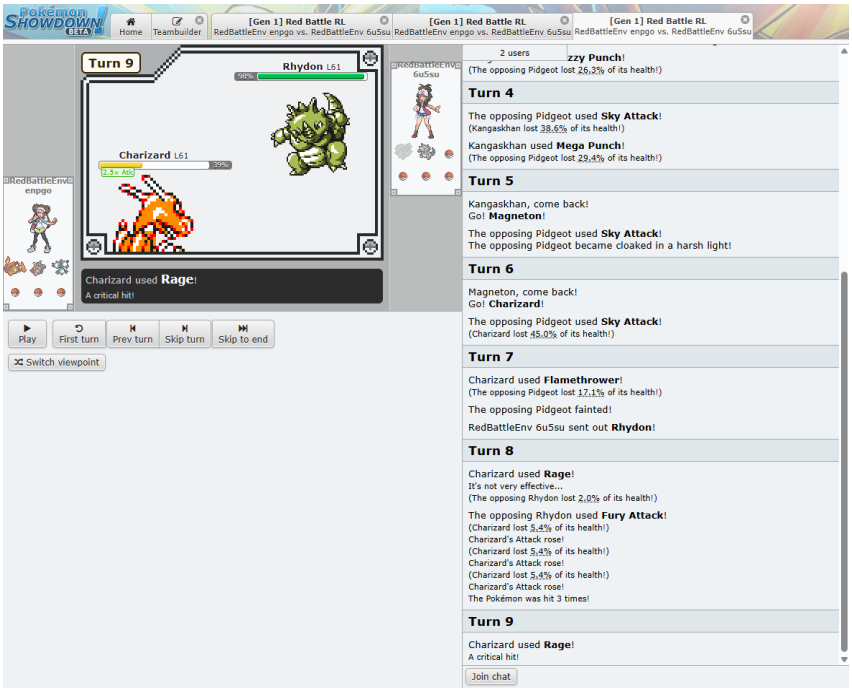


그림 18 Pokémon Showdown 환경

보상 설계

승리와 패배를 주요 보상으로 사용하였다. 다만 승패는 전투가 끝나야 확인되므로, 체력 변화와 포켓몬의 기절, 상태 이상과 능력치 변화도 중간 보상에 반영하여 매 턴의 행동이 학습에 반영되도록 하였다.

학습 환경

에이전트의 파티는 이상해씨, 파이리, 꼬부기 중 하나의 시작 포켓몬과 실제 게임에서 야생으로 만날 수 있는 포켓몬을 무작위로 조합하여 구성하였다. 상대 파티는 'Pokémon Red'에 등장하는 트레이너의 포켓몬 구성을 기준으로 하였으며, 야생 포켓몬과의 전투도 포함하였다. 또한 특정 지역이나 레벨의 전투에 학습이 치우치지 않도록 게임 진행 구간을 10 개의 섹션으로 나누고, 각 섹션에서 동일한 방식으로 학습하였다.

학습은 전투에서 얻은 상태, 행동, 보상을 이용하여 Actor 와 Critic 을 반복적으로 갱신하는 방식으로 진행하였다. 이를 통해 전투 상황에 따라 기술 사용과 포켓몬 교체를 선택하는 정책을 학습하였다.

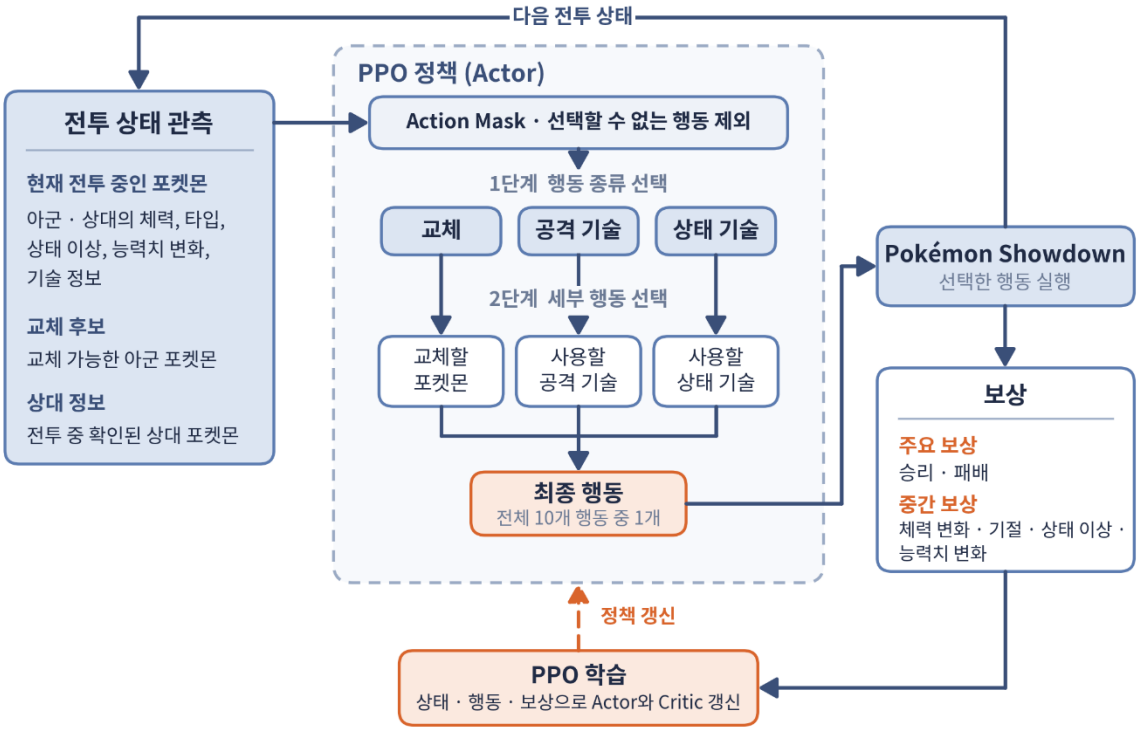


그림 19 2 단계 행동 선택 및 PPO 학습

실행

Pokémon Showdown 과 실제 'Pokémon Red'는 전투 정보를 제공하고 행동을 실행하는 방식이 다르다. Pokémon Showdown 에서는 poke-env 를 통해 체력과 기술 등의 정보를 바로 확인하고 선택한 행동을 명령으로 전달할 수 있지만, 실제 게임에서는 이러한 형태의 정보가 제공되지 않으며 방향키와 A, B 버튼으로 전투 메뉴를 직접 조작해야 한다.

이를 연결하기 위해 PyBoy 에서 전투와 관련된 메모리 값을 읽어, 학습에 사용한 관측 상태와 같은 형태로 변환하였다. 현재 포켓몬과 상대 포켓몬의 체력, 상태, 능력치, 기술

과 파티 정보를 메모리에서 확인하며, 학습된 정책은 변환된 정보를 입력받아 기술 사용 또는 포켓몬 교체 행동을 선택한다.

선택된 행동은 다시 버튼 입력으로 변환한다. 기술을 선택한 경우에는 전투 메뉴에서 FIGHT 로 이동한 뒤 해당 기술을 선택하고, 교체를 선택한 경우에는 PKMN 메뉴를 열어 교체할 포켓몬으로 커서를 이동한다. 각 입력 후에는 메모리의 메뉴 커서와 전투 상태를 확인하여 행동이 정상적으로 수행되었는지 판단한다.

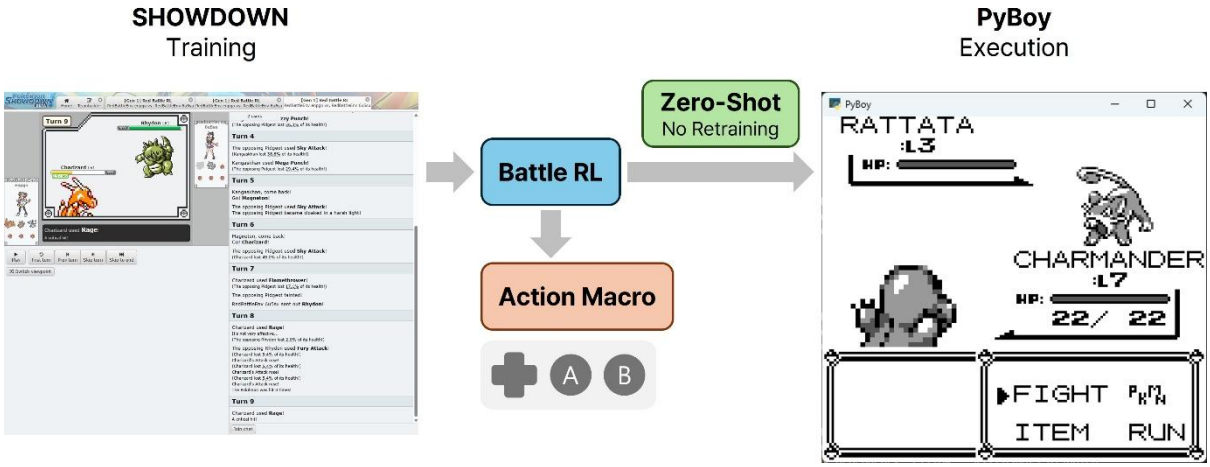


그림 20 Battle RL 정책의 PyBoy 적용 과정

3.3. LLM Planner

접근 방법

LLM Planner 는 'Pokémon Red'의 넓은 맵과 여러 퀘스트가 얽힌 장기 계획 문제를 다루기 위해, 게임 메모리에서 얻은 관측 정보를 바탕으로 다음에 도달할 목표 맵과 그곳에서 상호작용할 NPC, 그리고 도착한 뒤 수행할 스킬을 결정하는 상위 계획 역할을 수행한다.

언어 모델은 Claude Code CLI 의 비대화형 실행 모드로 호출하였으며, 모델은 Claude Sonnet 을 노력치 Low 설정으로 사용하였다. 호출은 이전 대화 맥락을 이어받지 않는 독립적인 구조로 동작하며, 호출할 때마다 현재 맵과 진행 이벤트, 관측된 맵과 NPC, 파티 상태를 프롬프트에 직접 주입하여 전달한다. 또한 웹 검색이나 파일 접근 등 외부 도구의 개입을 차단하여, 주입된 게임 내 관측 정보만을 근거로 계획을 세우도록 하였다. 호출은 한 사이클에 한 번만 이루어지며, 스킬이 실행되는 동안에는 추가로 개입하지 않는다.

프롬프트 구성

입력 프롬프트는 시스템 프롬프트와 사용자 프롬프트로 구분하였다. 시스템 프롬프트에는 언어 모델의 기본 역할, 게임의 최종 목표인 포켓몬 리그 도전, 출력 규칙, 목표 선택 기준과 출력 예시를 작성하였다. 이 내용은 표 3 과 같이 세 개의 파일로 나누어 저장하여, 실행 코드를 수정하지 않고도 목표 선택 규칙을 변경할 수 있게 하였다.

파일	내용
System.md	역할 정의, 최종 목표, 출력 형식, 지시할 수 없는 행동
Rules.md	목표 선택 절차와 판단 규칙
Fewshot.md	출력 형식 예시 및 반례

표 3 시스템 프롬프트의 구성

사용자 프롬프트는 호출 시점의 게임 상태를 정보로 변환한 것으로, 3.1 의 지도에서 읽어낸 값을 바탕으로 매 호출마다 새로 작성한다. 포함하는 정보는 표 4 와 같다.

게임 정보	내용	용도
현재 상태	현재 맵, 획득 배지 개수와 이름	다음 진행 위치 판단
관측한 맵	관측된 맵 이름과 각 맵의 탐험 종료 조건 충족 여부	목표 선택 범위
관측한 NPC	관측된 전체 맵에 대한 맵별 NPC 목록	목표 맵의 NPC 선택
최근 이벤트	발생한 이벤트 중 최근 10 개	직전 진행 상황 파악
파티 상태	포켓몬별 체력과 기술별 잔여 PP	회복 필요 여부 판단
이동 실패	직전 사이클의 Navigation 실패 사유	동일 목표 반복 방지
통행 차단 NPC	통행을 차단한 NPC 기록	진입 불가 경로 회피

표 4 사용자 프롬프트 내용

관측한 맵에는 이름과 함께 해당 맵의 탐험을 끝낼 수 있는지를 표시한다. 아직 볼 곳이 남은 맵과 더 볼 것이 없는 맵을 구분해 주어야, 도달하지 못한 장소로 향하는 길목을 고를 때 이미 다 확인한 맵을 목표로 지정하지 않기 때문이다.

NPC 정보는 현재 맵이 아니라 관측한 전체 맵에 대해 제공한다. 목표로 지정할 맵의 NPC 를 선택해야 하므로 현재 맵의 정보만으로는 판단할 수 없기 때문이다.

직전 이동 실패 정보는 다음 계획 1 회에만 포함하고 이후 제거한다. 실패 기록을 계속 유지하면 이후 경로가 열린 뒤에도 해당 목표를 영구적으로 회피하게 되기 때문이다. 통

행 차단 NPC 기록은 새로운 스토리 이벤트가 발생하면 삭제한다. 스토리가 진행되었다는 것은 차단 조건이 해제되었을 가능성을 의미하기 때문이다.

목표 선택 절차

계획의 목표는 반드시 관측된 맵 중에서 선택한다. 관측되지 않은 맵은 GlobalMap 에 노드로 등록되어 있지 않아 Navigation 이 경로를 생성할 수 없으므로, 실행 가능한 목표의 범위가 관측한 맵으로 제한된다.

그러나 이 제약을 그대로 적용하면 아직 도달하지 않은 지역에서 발생하는 이벤트는 진행할 수 없다. 이를 해결하기 위해 목표 선택을 다음 절차로 구성하였다.

1. 다음에 진행할 이벤트를 추론한다. 현재까지 획득한 배지와 발생한 이벤트를 근거로, 스토리상 다음에 일어나야 할 사건을 판단한다.
2. 그 이벤트가 발생하는 장소를 판단한다.
3. 그 장소가 관측 목록에 있는지 확인한다.
    - **있는 경우**: 해당 맵을 목표로 지정하고, 그 맵에서 관측된 NPC 중 이벤트 대상을 함께 지정한다.
    - **없는 경우**: 관측된 맵 중 해당 장소로 가는 경로상에 있을 것으로 판단되는 맵을 목표로 지정한다.

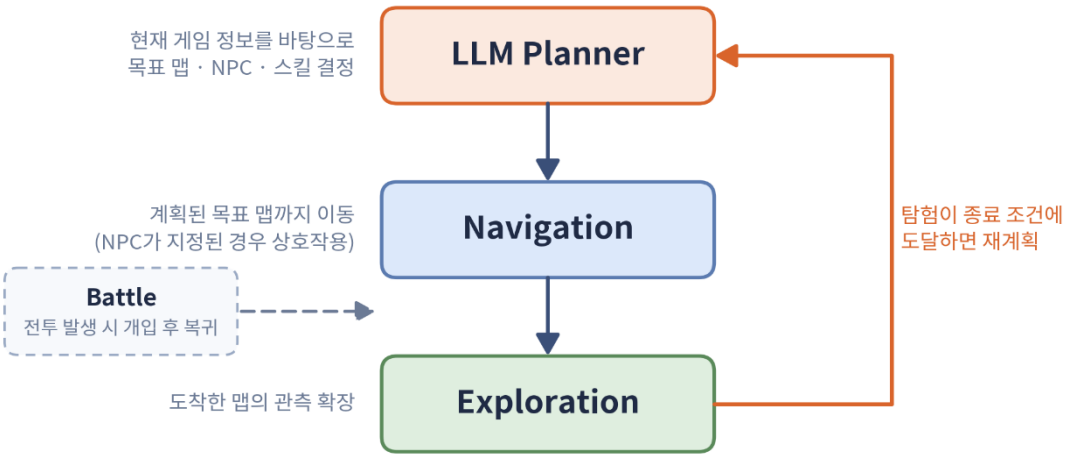


그림 21 아키텍처 실행 흐름

파티가 전투를 치를 수 없는 상태인 경우에는 위의 세 단계보다 먼저 회복을 목표로 정한다. 이때 회복을 별도의 항목으로 두지 않고, 관측된 포켓몬센터를 목표 맵으로 하고 그곳의 회복 담당 NPC 를 목표 NPC 로 지정하는 목표의 형태로 처리한다.

경유지를 지정하는 경우, 시스템은 아직 관측하지 않은 지역의 위치를 알 수 없으므로 그 선택은 언어 모델이 보유한 게임 사전 지식에 의존한다. 다만 목표 후보가 관측된 맵으로 고정되어 있어, 이 판단이 틀리더라도 실행할 수 없는 목표가 생성되지는 않는다. 어느 맵을 선택하든 Navigation 이 그 맵까지 이동한 뒤 Exploration 이 탐험을 수행하며, 이 과정에서 새로 발견된 맵은 GlobalMap 에 등록되어 다음 사이클의 프롬프트에 반영된다. 따라서 경유지 판단이 정확하지 않은 경우에도 관측 범위는 넓어지고, 최초 계획에서 지정할 수 없었던 목표를 이후 사이클에서 직접 지정할 수 있게 된다.

이러한 구조에서 LLM Planner 는 관측 범위 안에서 목표를 선택하고, Exploration 은 그 범위 자체를 확장한다. 계획에 부과된 제약이 Exploration 에 의해 사이클마다 완화되므로, 두 모듈의 반복을 통해 계획 가능 영역이 점진적으로 넓어진다.

출력 형식 및 사용

항목	내용	사용 여부
사용할 스킬	목표 맵에 도착한 뒤 수행할 스킬	실행
목표 맵	관측된 맵 중 이동할 장소 하나	실행
목표 NPC	목표 맵에서 상호작용할 NPC 또는 없음	실행
다음 해야 할 일	그 목표에서 일어나야 할 사건	기록
이유	해당 목표를 선택한 근거	기록

표 5 계획 출력 항목

목표 맵은 Navigation 이 이동할 맵을 정하는 데 사용하고, 목표 NPC 는 목표 맵에 도착한 후 상호작용할 대상을 지정한다. 사용할 스킬은 목표 맵에 도착한 뒤의 동작을 정한다. 이동 자체는 언제나 Navigation 이 담당하므로, 이 항목은 이동 수단을 고르는 것이 아니라 도착한 곳에서 NPC 와 상호작용할 것인지 아니면 그 맵의 관측을 넓힐 것인지를 정하는 것이다. 다음 해야 할 일과 이유는 실행 명령으로 사용하지 않으며, 계획의 판단 근거를 기록하여 사후 분석에 사용한다.

예외 처리

이동 중 전투가 발생한 경우에는 계획을 다시 요청하지 않는다. 전투는 언제든 발생할 수 있으므로 매번 상위 계획을 다시 세우면 사이클이 완료되지 않기 때문이다. Battle 이 전투를 처리한 뒤 중단되었던 스킬을 재개하되, Exploration 은 중단 지점에서 그대로 이어서 실행하고 Navigation 은 경로를 다시 계산한 뒤 재개한다.

반면 게임의 진행 상태를 바꾸는 이벤트가 발생하면 진행 중인 스킬을 중단하고 즉시 계획을 다시 요청한다. 이전 상태를 기준으로 세운 목표가 더 이상 유효하지 않을 수 있기 때문이다. 체육관 관장이나 라이벌과의 대결뿐 아니라 물건을 건네받는 것과 같이 전투를 수반하지 않는 이벤트도 여기에 해당하며, 일반 트레이너와의 전투 승리는 진행 상태를 바꾸지 않으므로 제외한다.



그림 22 주요 이벤트 예시

폐영역

관측이 더 이상 증가하지 않는 것만을 기준으로 탐험의 종료를 판단하면, 이미 모든 지역을 확인한 맵에서도 매 사이클마다 탐험을 시도하게 된다. 이를 방지하기 위해 맵 단위의 탐험 완료 조건인 폐영역 판정을 도입하였다.

폐영역은 현재 위치에서 갈 수 있는 범위를 모두 관측한 상태를 의미한다. 도달 가능한 영역의 경계가 벽, 물, 나무, 워프, 맵 경계로만 이루어지고 그 영역에 맞닿은 미관측 칸이 없으면 닫힌 것으로 본다. 이때 물과 나무는 파도타기와 풀베기를 아직 사용할 수 없을 때에 한하여 경계가 된다. 즉 이 판정은 맵 전체를 확인했다는 뜻이 아니라, 현재 능력으로 도달할 수 있는 곳을 모두 확인했다는 뜻이다.

경계를 이루는 워프에는 밟아서 확정한 워프와 관측으로만 확인한 문 후보가 모두 포함된다. Connection 워프의 경우 태초마을과 1 번 도로처럼 화면상으로는 이어져 보여 관측만으로 구분할 수 없으나, 그 좌표가 참조 자료에 있으므로 해당 칸이 격자에 기록된 순간 출구로 판정한다. 다만 이것만으로 판정하면 한 번도 나가 본 적 없는 맵이 관측만으로 완료 처리되므로, 실제로 통과한 출구가 최소 하나 있을 것을 함께 요구한다. 한편 동굴은 구멍 타일이 벽으로 분류되는 등 지형 규칙이 달라 판정이 잘못 성립할 수 있어 대상에서 제외하며, 정해진 스텝 예산 안에서만 탐험한다.

폐영역으로 판정된 맵은 Exploration 을 다시 호출하지 않으며, 이 정보는 사용자 프롬프트로 함께 전달된다. 다만 파도타기나 풀베기를 새로 사용할 수 있게 되면 경계였던 물과 나무 너머가 새로운 미관측 지역이 되므로, 이 경우 표시를 해제하고 다시 판정한다.

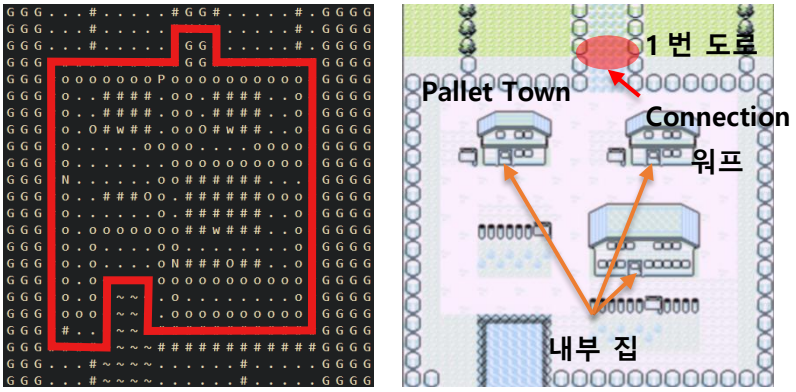


그림 23 포켓몬 Pallet Town 의 LocalMap 과 실제 맵 비교 사진

4. 연구 결과 분석 및 평가

4.1. Exploration Skill 성능 분석 및 평가

실험 개요

본 실험은 PPO 학습 과정에서 탐험 정책이 실제로 향상되는지 확인하고, 학습된 모델의 내부 및 외부 노드 확장 능력을 평가하는 것을 목적으로 한다. 이를 위해 서로 다른 모델을 동일한 시작 상태와 평가 조건에서 실행하여 성능 변화를 평가 및 비교하였다.

모델 평가 실험

모델 평가는 학습 지역과 미학습 지역을 포함한 여러 저장 상태에서 수행하였다. 모든 모델에는 동일한 시작 상태와 난수 시드, 최대 행동 스텝 및 환경 설정을 적용하여 평가 조건을 통제하였으며, 고정된 맵에서 각 시작 상태를 1,000 스텝 동안 3 회 반복 실행하였다. 다만, 맵을 실제로 탐험하지 않고 지나친 경우를 방문으로 인정하지 않기 위해, 해당 맵에 일정 스텝 이상 머문 경우에만 방문 노드로 집계하였다.

평가 지표	정의
Internal 확장률	방문한 Internal 노드 수 / 전체 Internal 후보 노드 수
External 확장률	방문한 External 노드 수 / 전체 External 후보 노드 수
전체 노드 확장률	방문한 Internal·External 노드 수 / 전체 후보 노드 수
정체 비율	진행이 없어 패널티가 발생한 스텝 수 / 전체 스텝

표 6 Exploration 주요 평가지표 정의

여기서 Internal 확장률은 Target 에 직접 연결된 건물과 내부 공간 중 방문한 노드의 비율이고, External 확장률은 Target 경계 밖의 평가 대상 노드 중 방문한 비율이다.

단일 모델의 성공 기준은 전체 노드 확장률 60% 이상으로 설정하였다. 평가 시작 상태에 따라 실제로는 방문할 수 없는 노드도 후보에 포함되므로, 100%를 기준으로 삼을 수 없기 때문이다. 세부적으로는 Internal 확장률과 External 확장률을 함께 확인하였다. 결과는 각 평가 에피소드의 평균과 표준편차로 제시하여, 확률적 행동 선택에 따른 성능 편차를 함께 나타내었다.

모델 비교 실험

모델 평가를 통과한 모델들 사이의 성능을 비교하기 위해 비교 실험을 수행하였다. 평가 실험과 동일한 환경 설정을 사용하였으며, 학습 과정에서 기록된 지표를 이용하여 비교하였다.

비교 지표는 스텝에 따른 Internal 확장률, External 확장률, 전체 노드 확장률, 정체 비율이며, 각 에피소드의 평균값을 사용하였다. 노드 확장률은 높을수록, 정체 비율은 낮을수록 우수한 것으로 판단하였다.

model_000150 불필요한 행동 제거를 위해 penalty scale 이 높은 모델
model_000072 penalty 비중을 줄이고 내부 탐색 리워드 scale 을 높인 모델
model_00006 외부 탐색 리워드와 내부 탐색 리워드 scale 을 모두 높인 모델
model_000027 방문 타일에 대한 리워드를 관측한 타일에 대한 리워드와 비슷한 수준으로 높인 모델

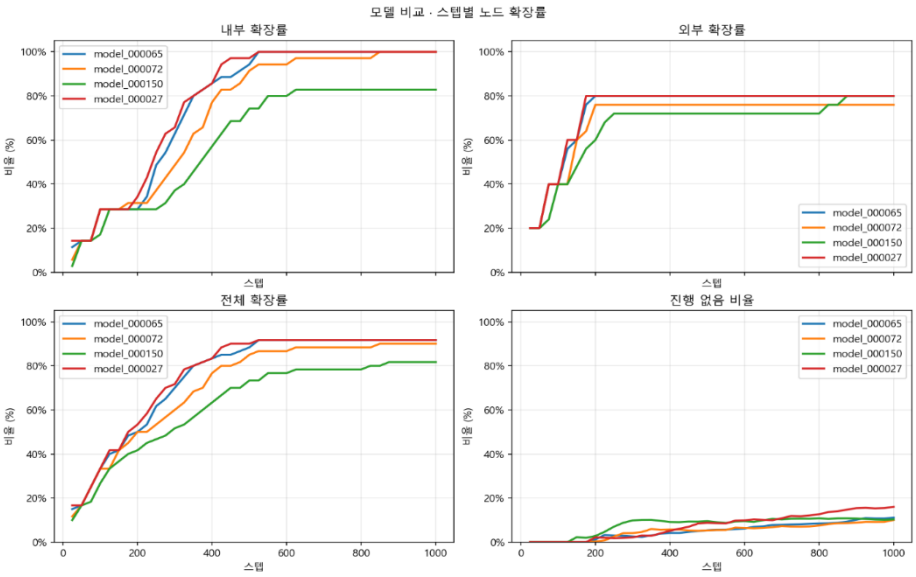


그림 24 모델 비교 및 스텝별 노드 확장률

penalty scale 을 높여 학습한 model_000150 은 전체 확장률에서 가장 낮은 성능을 기록하였다. 다만 정체 비율은 평가 초반에 다소 빠르게 상승한 뒤 1,000 스텝에 이르기까지 10% 내외를 유지하여 최종 9.7%로 낮게 나타났다. 그러나 탐험의 목적이 관측 범위의 확장에 있으므로 전체 확장률을 우선 기준으로 판단하였으며, 이에 따라 비교적 낮은 성능의 모델로 보았다.

내부 탐험 보상의 비중을 높인 model_000072 는 내부 노드 확장률에서 model_000150 보다 18% 높은 100%를 기록하였다. 외부 노드 확장률은 76%로 5% 낮았으나 전체 확장률에서는 8.3% 높은 수치를 보였다. 이는 패널티의 비중을 높이는 것보다 탐색 보상을 높이는 편이 더 나은 성능으로 이어질 수 있음을 시사한다.

model_000065 는 model_000072 와 달리 외부 탐색 보상도 함께 높인 모델로, 노드 확장률 지표에서 모두 가장 높은 수치를 기록하였다. 내부 노드 확장률은 100%, 외부 노드 확장률은 80%였으며, 정체 비율 또한 9.6%로 model_000150 보다 0.1% 낮았다.

탐험 보상은 방문한 타일에 대한 보상과 관측한 타일에 대한 보상으로 구분된다. model_000027 을 제외한 모델은 방문 대비 관측 보상의 비율이 약 3:7 로 학습되었으나, model_000027 은 5:5 비율로 학습하였다. 그 결과 model_000027 은 1,000 스텝 시점의 노드 확장률에서 model_000065 와 동일한 수치를 기록하였으나, 확장 속도는 더 빨랐다. 다만 정체 비율은 다른 모델보다 약 5% 높게 나타났다.

이상의 결과로부터 두 가지를 확인할 수 있다. 첫째, 패널티를 강화하는 것보다 보상을 강화하는 방식이 탐험 성능 향상에 더 효과적이다. 둘째, 관측 보상은 에이전트가 정체 없이 움직이도록 하는 반면 방문 보상은 노드 확장 속도를 높이는 경향을 보인다. 따라서 두 보상의 비율을 적절히 조정하는 Reward Shaping 이 탐험 정책의 학습에 중요한 요소로 판단된다.

4.2. Battle Skill 성능 분석 및 평가

평가 환경 및 방법

Battle 정책의 전투 수행 성능을 확인하기 위해 Pokémon Showdown 기반 환경에서 최종 모델을 평가하였다. 평가 대상은 게임 진행 구간에 따라 구분한 10 개 섹션(S0~S9)으로 구성하였으며, 각 섹션에서 100 회씩 총 1,000 회의 전투를 수행하였다. 평가에는 트레이너 전투와 야생 포켓몬 전투를 모두 포함하였고, 전체 승률과 전투 유형별 승률, 진행 구간별 승률을 분석하였다.

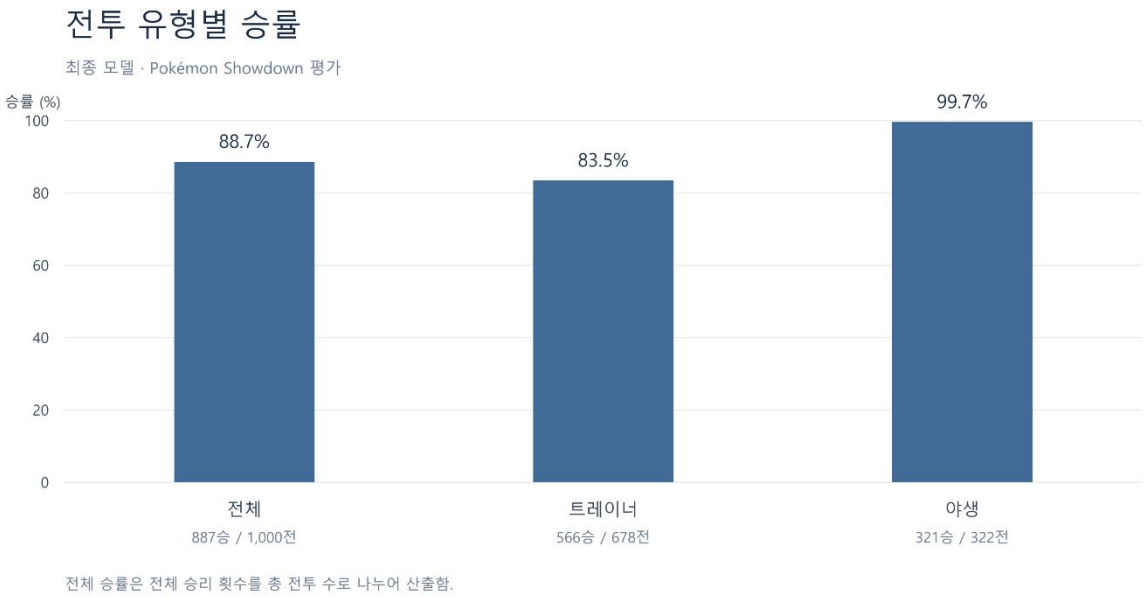


그림 25 전투 유형별 승률

전체 및 전투 유형별 성능

총 1,000 회의 전투에서 887 승 113 패를 기록하여 전체 승률은 88.7%로 나타났다. 전투 유형별로는 트레이너 전투 678 회 중 566 승을 기록하여 83.5%, 야생 전투 322 회에서 321 승을 기록하여 99.7%의 승률을 보였다.

전체 패배 113 회 중 112 회가 트레이너 전투에서 발생하였다. 따라서 트레이너 전투에서 선택한 기술과 교체 시점 등을 살펴보고, 패배 원인을 확인할 필요가 있다.

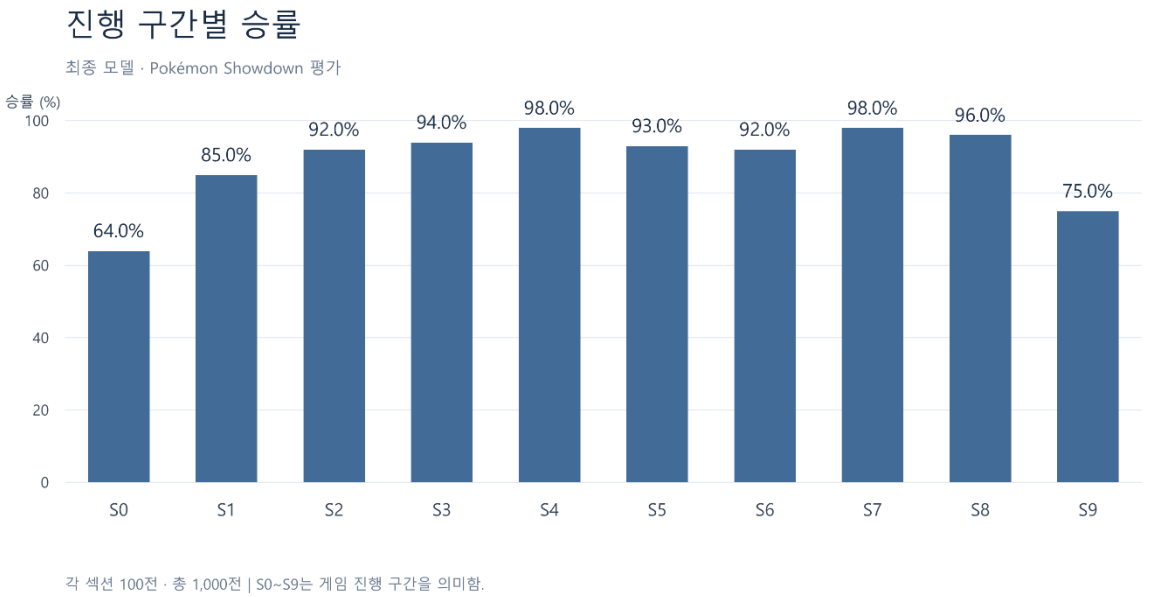


그림 26 진행 구간별 승률

게임 진행 구간별 성능

S2~S8 에서는 92~98%의 승률을 기록한 반면, S0 은 64%, S1 은 85%, S9 는 75%로 나타났다. 특히 S0 과 S9 에서 상대적으로 낮은 성능을 보였으므로, 해당 구간의 파티 구성과 전투 조건을 중심으로 실패 사례를 분석할 필요가 있다. 각 구간의 승률은 100 회의 전투에서 얻은 결과이므로, 반복 평가에서도 비슷한 결과가 나타나는지 확인할 필요가 있다.

실제 게임 적용 및 한계

학습된 정책을 PyBoy 기반 Pokémon Red 에 연동하여 전투 동작을 확인하였다. 그러나 전투 횟수와 승패를 별도로 집계하지 않아 실제 게임 환경의 승률은 산출하지 않았다. 향후에는 PyBoy 전투 기록을 자동으로 수집하여 승률을 측정하고, 정책의 판단 오류와 게임 입력 과정의 실행 오류를 구분하여 개선할 필요가 있다.

4.3. 통합 실행 결과

실험 방법

연구 구조로 제시한 Map, Skills, LLM Planner 를 통합하여, 사람의 개입 없이 게임이 어느정도 진행되는 지 확인하였다. 시작 지점은 오박사로부터 첫 포켓몬을 받은 직후의

태초 마을 풀숲 앞으로 설정하였으며, 이 상태에서 그림 21 과 같은 흐름으로 자동 진행하였다. 연구 범위 밖의 과제(기술머신 사용, 아이템 습득 및 사용)들은 앞서 설명한 대로 스킵하였다. 측정 항목은 각 스토리 이벤트를 몇 스텝에 달성하였는지와, 관측 확장에 관한 지표이다.

진행 결과

실행 결과 에이전트는 사람의 개입없이 포켓몬스터 첫번째 관장을 넘어 달맞이산까지 진행하였다.

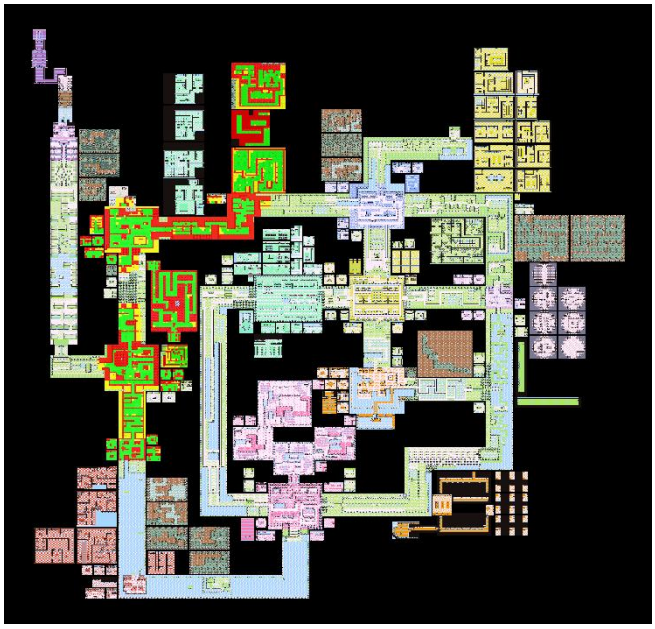


그림 27 Pokemon-Playing Agent 의 실제 진행 지도

이벤트별 달성 스텝(시점)은 아래 표와 같다.

진행 순서	이벤트 이름	걸린 시간	달성 스텝
1	Got Oaks Parcel	2 분 53 초	1690
2	Got Pokedex	3 분 9 초	1788
3	Oak Got Parcel	3 분 9 초	1788
4	1St Route22 Rival Battle	3 분 13 초	1789
5	Route22 Rival Wants Battle	3 분 13 초	1789
6	Beat Route22 Rial 1St Battle	7 분 7 초	8275
7	Beat Brock	24 분 31 초	40593

표 7 이벤트별 달성 스텝

관측 측면에서는 총 24 개의 맵이 GlobalMap 에 등록되었으며, 이 중 18 개가 폐영역으로 판정되어 이후 탐험 대상에서 제외되었다. 계획 측면에서는 총 200 회의 사이클이 수행되었다.

5. 결론 및 향후 연구 방향

5.1. 결론

본 연구에서는 'Pokémon Red'와 같이 서로 다른 성격의 과제가 연속적으로 발생하고 이벤트 간 선후관계가 존재하는 환경에서, 하나의 정책이 게임 전체를 학습하는 단일 에이전트 방식으로는 이를 다루기 어렵다는 점에 주목하여 계층형 에이전트 구조를 제안하였다.

게임 진행을 하나의 학습 문제로 다루는 대신, 탐험과 목표 지점 이동, 전투의 세 가지 과제로 분해하고 각 과제에 성격에 맞는 방법을 적용하여 스킬 간의 간섭을 해소하였다. 지도가 확보된 공간에서의 이동은 최적해를 계산할 수 있으므로 탐색 알고리즘을, 무엇이 최선인지 미리 정의하기 어려운 탐험과 전투에는 강화학습을 적용하였다. 또한 다음에 무엇을 할지 결정하는 상위 판단을 LLM Planner 로 분리함으로써, 이벤트마다 보상을 설계하지 않고도 진행에 필요한 판단을 수행할 수 있도록 하였다.

이때 상위 계층과 하위 스킬이 공통으로 참조할 정보로 Global Map 과 Local Map 의 구조를 사용하여, 맵 사이의 이동과 맵 내부의 이동을 분리하였다. 이 구조에서 LLM Planner 는 관측된 맵으로 목표 범위가 제한되며, 이 제약은 Exploration 이 관측을 확장함에 따라 사이클마다 완화된다. 즉 계획 가능 영역이 실행 결과에 따라 점진적으로 확대되는 구조를 구성하였다. 상위 계층이 사전에 주어진 지도가 아니라 하위 계층이 실제로 관측한 정보만을 근거로 판단하도록 한 것이 이 구조의 핵심이다.

이처럼 본 연구의 의의는 상위의 목표 판단과 하위의 실행 스킬을 분리하고, 그 사이를 관측된 정보만으로 연결하는 구조를 실제로 동작하는 형태로 구성하였다는 점에 있다. 이 구조에서 상위 계층은 하위 스킬의 구현 방식을 알 필요가 없고, 하위 스킬 또한 전체 진행의 맥락을 알 필요가 없다. 따라서 하위 스킬을 해당 도메인의 것으로 교체하면 같은 구조를 다른 문제에도 적용할 수 있다. 로봇의 장기 복합 과제 역시 선행 조건을 만족시키며 성격이 다른 여러 스킬을 조합해야 한다는 점에서 구조적으로 대응되므로, 본 연구의 계층형 의사결정 구조를 적용할 수 있을 것으로 기대한다. 다만 본 연구는 게임 환경에서만 검증을 수행하였으므로, 실제 로봇 환경에서의 적용 가능성은 추가적인 실험을 통해 확인해야 한다.

5.2. 향후 연구 방향

유동적인 Skill 호출 구조

현재 실행 사이클은 Planning 에서 Navigation 을 거쳐 Exploration 으로 이어지는 고정된 순서로 구성되어 있다. 이 구조에서는 이동만 필요한 상황에서도 탐험이 실행되고, 반대로 추가 탐험이 필요한 상황에서 사이클이 종료된다. 향후에는 LLM Planner 가 목표뿐 아니라 수행할 스킬과 그 순서까지 결정하는 구조로 확장할 필요가 있다. 이를 위해서는 각 스킬의 실행 조건과 종료 조건을 명시적으로 정의하고, 스킬의 실행 결과를 다음 계획의 입력으로 전달하는 구조가 필요하다.

Exploration Skill 의 분할

현재의 Exploration 은 성격이 다른 두 행동을 하나의 정책으로 학습한다. 하나는 현재 맵의 격자를 넓혀 관측을 확장하는 것이고, 다른 하나는 아직 통과하지 않은 워프로 진입하여 새로운 맵을 발견하는 것이다. 그러나 워프를 통과하는 순간 맵이 전환되어 격자를 넓힐 대상 자체가 사라지므로, 두 행동은 한 에피소드 안에서 목적이 충돌한다. 보상의 관점에서도 맵 전환 한 번이 새로 관측한 수십 개의 칸과 맞먹는 크기를 가지게 되어, 격자 확장에 대한 보상이 상대적으로 묻히게 된다. 이에 따라 관측을 넓히는 Coverage 와 아직 통과하지 않은 워프로 진입하는 SeekWarp 로 탐험을 분할하는 방안을 검토하고 있다. 두 스킬은 종료 조건과 성공의 기준이 서로 다르므로, 분할을 통해 각 정책이 하나의 목적만을 최적화하도록 만들 수 있을 것으로 본다.

스킬의 확장

본 연구의 범위 밖으로 둔 아이템 관리와 기술머신 사용 등의 과제는 현재 사전에 정의된 규칙 기반 스크립트로 처리하고 있다. 그러나 스크립트는 절차를 미리 모두 기술해야 하고 상황이 조금만 달라져도 대응하지 못하므로, 과제 계층 구조에서 상위에 위치한 과제일수록 이러한 방식으로 다루기 어렵다. 향후에는 이들 과제 또한 스킬의 형태로 구성하여 LLM Planner 가 직접 지시할 수 있는 대상에 포함할 필요가 있다. 이 경우 플래너가 선택할 수 있는 행동의 범위가 넓어지므로, 선행 조건을 만족시키는 순서를 스스로 구성할 수 있는지도 함께 확인할 수 있을 것이다.

6. 구성원별 역할 및 개발 일정

업무	5월				6월				7월				8월				9월				10월
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1
기존 연구 분석 및 문제 재정의																					
시스템 구조 설계																					
Dual Map 구현 및 Navigation 구현																					
Exploration RL 구현 및 학습																					
Battle RL 학습 환경 구축 및 정책 학습																					
LLM Planner 구현 및 목표 설정 기능 연동																					
이동·탐색·전투 모듈과 LLM Planner 통합																					
통합 동작 테스트 및 최대 진행 단계 확인																					
실패 사례 분석																					
최종 보고서 작성 및 발표 준비																					

7. 참고 문헌

논문 내용에 직접 관련이 있는 문헌에 대해서는 관련이 있는 본문 중에 참고문헌 번호를 쓰고 그 문헌을 참고문헌란에 인용 순서대로 기술한다. 참고문헌은 영문으로만 표기하며 학술지의 경우에는 저자, 제목, 학술지명, 권, 호, 쪽수, 발행년도의 순으로, 단행본은 저자, 도서명, 발행소, 발행년도의 순으로 기술한다.

〈단행본 예시〉

[1] R. Bellman. *Introduction to Matrix Analysis*, 2nd Ed., pp. 234. McGraw-Hill, New York, 1979.

[1] Statista, “AI robotics market size worldwide 2020-2032.” https://www.statista.com/forecasts/1449851/ai-robot-market-worldwide

[2] Mordor Intelligence, “Physical AI Market Size & Share Analysis - Growth Trends and Forecast(2026-2031),” 2026.
https://www.mordorintelligence.com/industry-reports/physical-ai-market
[3] M.Pleines, D.Addis, D.Rubinstein, F.Zimmer, M.Preuss, P.Whidden, “Pokémon Red via Reinforcement Learning,” arXiv:2502.19920, 2025.
[4] D. Rubinstein, K. Donovan, D. Addis, K. W. Choe, J. Suarez, P. Whidden, "Learning Pokémon With Reinforcement Learning," 2025. https://drubinstein.github.io/pokerl/

〈논문지 예시〉

[1] K. Park, H. Hwang, C. Lee, and S. Min, "Analysis of Delay-Bandwidth Normalization Characteristic in Decay Usage Algorithm of UNIX," *Journal of KIISE : Computer Systems and Theory*, Vol. 34, No. 10, pp. 511-520, Oct. 2007. (in Korean)
[2] D. Kim, H. Kim, and J. Seo, "A Statistical Prediction Model of Speakers' Intentions in a Goal-Oriented Dialogue," *Journal of KIISE : Software and Applications* , Vol. 35, No. 9, pp. 554-561, Oct. 2008. (in Korean)
[3] D. O. Seong, M. H. Yeo, and J. S. Yoo, "An Energy Efficient Continuous Skyline Query Processing Method in Wireless Sensor Networks," *Journal of KIISE : Computing Practices and Letters*, Vol. 15, No. 4, pp. 275-279, Apr. 2009. (in Korean)
[4] J. Kim, I. Song and M.-H. Kim, "An Efficient Keyword Search Method on RDF Data," *Journal of KIISE : Databases*, Vol. 35, No. 6, pp. 495-504, Dec. 2008. (in Korean)
[5] J. Choe and D. Yoo, "A Mechanism for Handling Selfish Nodes using Credit in Sensor Networks," *Journal of KIISE : Information Networking*, Vol. 35, No. 2, pp. 120-129, Oct. 2008. (in Korean)
[6] T.-Q. Nguyen, J.-S. Heo, J.-H. Lee, Y.-R. Kim, and K.-Y. Whang, "Query Expansion Using Augmented Terms in an Extended Boolean Model," *Journal of Computing Science and Engineering*, Vol. 2, No. 1, pp. 26-43, Mar. 2008.

〈학술대회 발표 논문집 예시〉

[1] S. Yu, S. J. Kim, "A Design of a Context-Aware Fault Recovery System for Smart Homes," *Proc. of the KIISE Korea Computer Congress 2008*, pp. 354-358, 2008. (in Korean)
[2] K. J. Kang, S. J. Kim, "A Design of Probability-based Knowledge Model for Environment Control Services on Smart Homes," *Proc. of the 35th KIISE Fall Conference*, pp. 291-358, 2008. (in Korean)

〈WEB Site 예시〉

[1] J. Jones. (1991, May 10). Networks (2nd ed.) [Online]. Available: http://www.atm.com (downloaded 2012, Sep. 10)